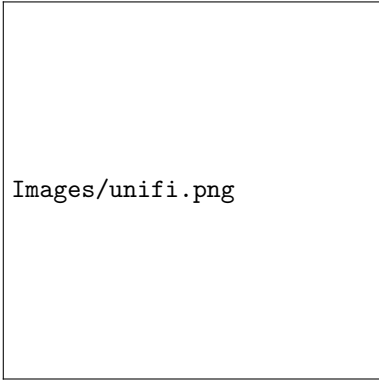




Images/unifi.png

UNIVERSITÀ DEGLI STUDI DI FIRENZE
SCUOLA DI INGEGNERIA - DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Tesi di Laurea Triennale in Ingegneria Informatica

Controllo di copertura dinamico per il monitoraggio cooperativo in tempo reale

Relatore

Giorgio Battistelli

Correlatore

Nicola Forti

Candidato

Ivan Necerini

Anno accademico 2023 / 2024

Indice

Introduzione	5
1 Descrizione del problema	7
1.1 Formulazione matematica del problema	8
1.2 Soluzioni proposte	10
2 Algoritmi	13
2.1 Algoritmo di coverage V1: algoritmo di ascesa del gradiente classico . .	15
2.2 Algoritmo V2: aggiunta di un moto Browniano	16
2.3 Algoritmo V3: aggiunta di un potenziale repulsivo	19
3 Implementazione degli algoritmi	22
3.1 Operazioni preliminari	23
3.2 Descrizione del codice (e tabelle con pseudocodice)	27
3.2.1 Versione 1	27
3.2.2 Versione 2	28
3.2.3 Versione 3	30
3.3 Librerie Python utilizzate	32
3.3.1 NumPy	32
3.3.2 Pickle	33
3.3.3 Matplotlib	33
3.3.4 Pytorch	33
4 Simulazione e discussione dei risultati	36
4.1 Setting dei parametri iniziali	36
4.2 Analisi delle operazioni preliminari	39
4.3 Analisi e grafici dei dati sperimentali	42
4.4 Richiesta di coverage meno restrittiva	51
Conclusioni	59
Bibliografia	61
Ringraziamenti	64

Introduzione

Il *controllo di copertura dinamico* è un problema centrale in molti campi applicativi, tra cui la robotica, la sorveglianza e la gestione delle emergenze. L'obiettivo principale di questo problema è quello di garantire che un'area di interesse sia monitorata in modo efficace e continuo da una flotta di agenti mobili, come droni o veicoli autonomi, equipaggiati con sensori. *Questo compito diventa particolarmente complesso e, interessante e pratico quando l'ambiente è dinamico e i punti di interesse (targets) sono in movimento.* Il controllo dinamico della copertura è dunque un tipo di controllo cooperativo che richiede a un sistema multi-agente di monitorare dinamicamente un'area di interesse nel tempo (2).

Nella letteratura, esistono due principali categorie di controllo della copertura: statica e dinamica. Il controllo della copertura statica si occupa di determinare il posizionamento ottimale degli agenti in modo che le aree di interesse siano completamente coperte dall'unione delle zone di rilevamento degli agenti. Al contrario, il controllo della copertura dinamica sfrutta la mobilità degli agenti per monitorare continuamente le aree di interesse nel tempo. In altre parole, ogni area deve essere monitorata da agenti mobili per un periodo sufficiente a garantire una copertura efficace. Il controllo della copertura statica, tuttavia, presuppone implicitamente la disponibilità di un numero sufficiente di agenti o la presenza di aree di interesse di dimensioni ridotte. In caso contrario, non vi è alcuna garanzia che tutte le aree possano essere completamente coperte. Queste limitazioni possono essere superate con il controllo della copertura dinamica, che permette il monitoraggio di vaste aree di interesse utilizzando un numero ridotto di agenti.

Tuttavia, sia nel controllo della copertura statica che dinamica, è spesso necessario disporre di modelli dell'ambiente e della cinematica/dinamica degli agenti per sviluppare leggi di controllo del movimento efficaci (2).

In questo lavoro, ci concentriamo sull'implementazione di un algoritmo di controllo di copertura basato su una *strategia greedy*. Questo tipo di algoritmo si distingue per prendere decisioni in base all'informazione disponibile nell'istante attuale, senza fare previsioni o ottimizzazioni a lungo termine. L'obiettivo principale è massimizzare l'indice di copertura totale in ogni momento, reagendo rapidamente ai cambiamenti dell'ambiente e delle posizioni dei targets.

Un *algoritmo greedy* (o avido) opera secondo il principio di fare la scelta che sembra essere la migliore al momento, con la speranza che questa scelta locale porterà a una soluzione globale ottimale. La strategia greedy è caratterizzata da una serie di scelte sequenziali, ognuna delle quali seleziona l'opzione che offre il miglior beneficio immediato. Questo approccio è spesso efficace per risolvere problemi di ottimizzazione combinatoria e ha il vantaggio di essere semplice da implementare e computazionalmente efficiente (22).

Tuttavia, la natura miope della strategia greedy significa che essa non sempre produce una soluzione ottimale globale. Nonostante questo, in molti casi pratici, gli algoritmi greedy forniscono soluzioni di buona qualità in tempi ragionevoli, il che li rende adatti per applicazioni in tempo reale come il controllo di copertura dinamica (23). Nel contesto del controllo di copertura, un algoritmo greedy seleziona la prossima posizione per ogni agente in modo da massimizzare l'aumento immediato dell'indice di copertura totale.

L'algoritmo di ascesa del gradiente è stato scelto per la sua capacità di ottimizzare direttamente una funzione obiettivo, in questo caso l'indice di copertura totale, che quantifica la qualità della copertura fornita dagli agenti sui targets. Questo metodo è stato esteso per considerare le limitazioni pratiche, come le capacità di rilevamento e comunicazione limitate degli agenti, nonché la necessità di mantenere una copertura sufficiente su tutti i targets durante l'intera durata della missione.

Modelli accurati possono essere difficili da ottenere in molte applicazioni pratiche. Pertanto, questo lavoro sviluppa un approccio model-free, dove gli agenti apprendono direttamente dalle interazioni con l'ambiente per ottenere un controllo dinamico della copertura.

Il presente lavoro mira a sviluppare e implementare un algoritmo di controllo di copertura dinamico che sfrutti i principi dell'ottimizzazione con strategia greedy e dell'ascesa del gradiente. Gli agenti mobili (droni) devono monitorare una serie di targets mobili, garantendo che ogni target sia coperto cumulativamente a un livello desiderato nel tempo. La formulazione matematica del problema, le soluzioni proposte e i dettagli dell'implementazione dell'algoritmo saranno discussi nelle sezioni successive.

1 Descrizione del problema

In questo lavoro, un piccolo gruppo di agenti mobili (*UAVs*, *droni*) con capacità di rilevamento limitate, posti ad **un'altezza fissa**, ha l'obiettivo di monitorare dinamicamente un'area popolata da un gran numero di punti di interesse (*Targets mobili*). Lo scenario di riferimento potrebbe essere, ad esempio, un'area portuale dove i targets sono rappresentati da imbarcazioni che si muovono continuamente nel tempo.

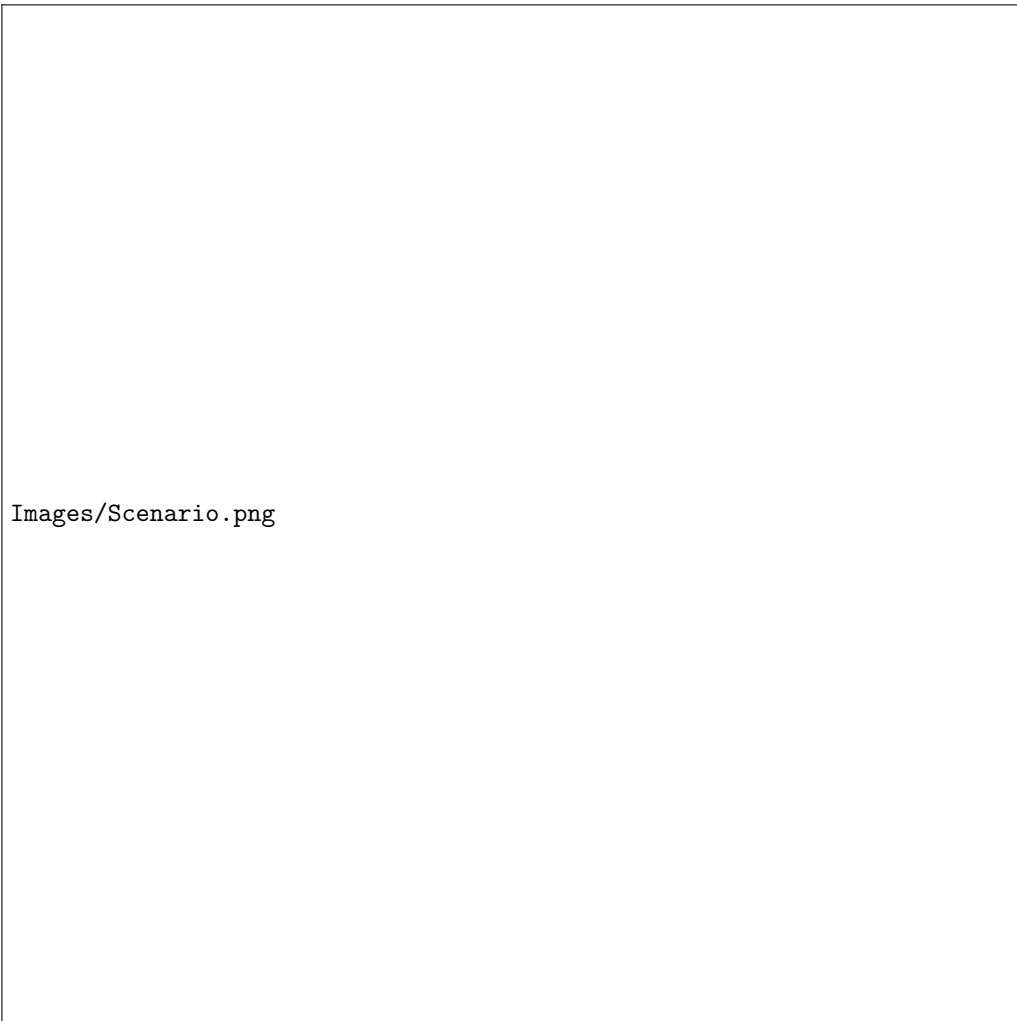


Figura 1: Possibile scenario di un controllo della copertura dinamico (2).

Il monitoraggio dinamico di un'area di interesse richiede che, istante per istante, i droni forniscano una visione accurata (o almeno sufficiente) di tutti i target mobili presenti. L'obiettivo generale può variare, come ad esempio l'evitare collisioni tra le imbarcazioni.

Per raggiungere questo obiettivo, è fondamentale garantire una *copertura sufficiente* dell'area. Nel contesto di questo esperimento, le traiettorie dei target saranno predefinite e verranno assegnate posizioni iniziali ai droni. **Il compito consiste dunque nello scrivere un algoritmo di copertura per determinare le traiettorie dei droni in modo tale che, al termine delle**

traiettorie dei target, venga raggiunto un grado sufficiente di copertura dell'area. (vedi 2).

I droni dovranno quindi "seguire" i target, assicurando una buona visione dell'area di interesse per tutta la durata dell'esperimento. La definizione dettagliata del *grado di copertura sufficiente* verrà fornita nella sezione 1.2 della tesi.

1.1 Formulazione matematica del problema

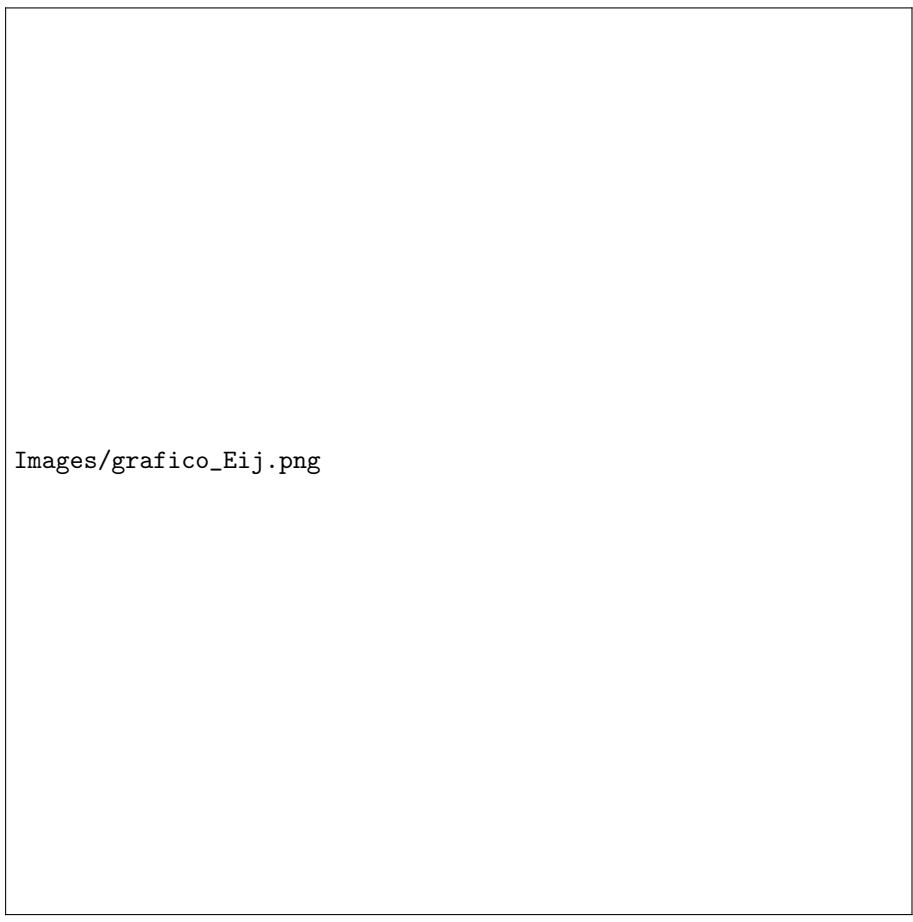
Consideriamo M punti di interesse (***Targets dinamici***) distribuiti in uno spazio di lavoro bidimensionale W . Le posizioni dei targets sono denotate da $q_j(t) \in \mathbb{R}^2$, con $j \in \{1, \dots, M\}$. Un gruppo di N ***Agenti mobili, dinamici*** (*Agents*), ad un'altezza fissata, è incaricato di monitorare continuamente questi targets. Le posizioni degli agenti sono denotate da $p_i(t) \in \mathbb{R}^2$, con $i \in \{1, \dots, N\}$.

Si presume che ogni agente abbia una zona di rilevamento limitata, modellata da un'area discoidale con un certo *raggio di copertura* $r \in \mathbb{R}^+$ centrata su se stesso.

Poiché la qualità del rilevamento generalmente degrada con la distanza dal target, il rilevamento basato sulla distanza dell'agente i sul target j è caratterizzato da una ***funzione di misurazione***:

$$E_{ij}(p_i(t), q_j(t)) = \begin{cases} M_P \left(\frac{(l_{ij}(t) - r)^2}{r^4} \right), & \text{se } l_{ij}(t) \leq r^2, \\ 0, & \text{se } l_{ij}(t) > r^2, \end{cases} \quad (1)$$

dove $l_{ij}(t) = \|p_i(t) - q_j(t)\|^2$ è la distanza, all'istante t , tra il drone i e il target j , e $M_P \in \mathbb{R}^+$ è una qualità di rilevamento di picco. Il modello di rilevamento indica che la qualità del rilevamento del target j raggiunge il picco quando l'agente i coincide con esso, e diminuisce monotonamente quando l'agente i si allontana da esso (2).



Images/grafico_Eij.png

Figura 2: Funzione di misurazione, $mp=1$, $r=150$.

1.2 Soluzioni proposte

In questa sottosezione viene definita la procedura matematica che stabilisce il *grado di copertura sufficiente*, menzionato nel paragrafo 1.

Il controllo della copertura efficace basato su un algoritmo di ascesa del gradiente, con strategia greedy, in questo lavoro ha l'obiettivo di guidare gli agenti mobili affinché garantiscano che ogni target nella regione di interesse sia coperto cumulativamente a un livello desiderato nel tempo (2). Per ogni target j , viene definito un **indice di copertura** $E_j(t)$, che quantifica la qualità della copertura al tempo t fornita dai vari agenti. Ogni agente contribuisce all'indice di copertura in base alla propria vicinanza al target, secondo la seguente funzione di misurazione:

$$E_j(t) = \sum_{i=1}^N E_{ij}(p_i(t), q_j(t)), \quad j \text{ fisso} \quad (2)$$

Come indicato nella sezione 1.1, la funzione di misurazione $E_{ij}(p_i(t), q_j(t))$ varia nell'intervallo $[0, M_P]$. Di conseguenza, l'indice di copertura $E_j(t)$ apparterrà all'intervallo $[0, M_P \cdot N]$.

Successivamente, viene definito un **indice di copertura totale** $E(t)$, espresso come:

$$E(t) = \sum_{j=1}^M \sigma(E_j(t)) \quad (3)$$

dove:

$$\sigma(E_j(t)) = \frac{\tanh(E_j(t) - E^*) + 1}{2} \quad (4)$$

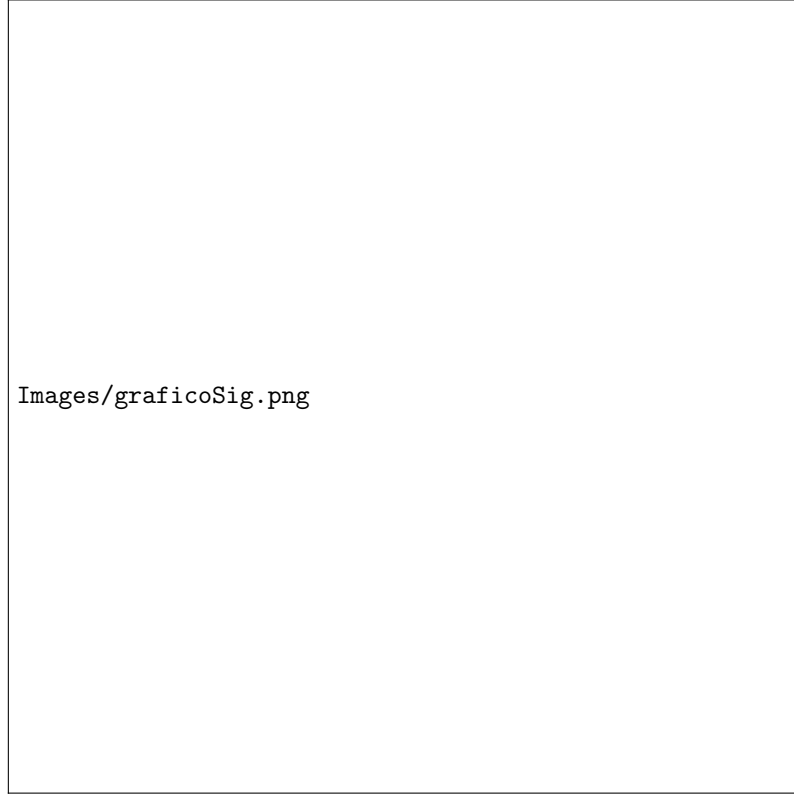


Figura 3: Funzione sigmoideale, $E^* = 2$

La definizione di queste funzioni e indici di copertura è fondamentale per raggiungere l'obiettivo del problema. La soglia E^* rappresenta un *lower bound index (LB)*, ovvero ogni indice di copertura di un target generico j al tempo t ($E_j(t)$) deve essere maggiore di questa soglia per considerare "accettabile" la copertura fornita dagli agenti. È importante notare che ***i parametri sono fortemente interdipendenti***; infatti, in base a M_P , varia l'intervallo in cui è contenuta la funzione di misurazione $E_{ij}(p_i(t), q_j(t))$ e, di conseguenza, anche gli indici di copertura ($E_j(t)$). È quindi necessario settare i vari parametri caso per caso. Nella simulazione effettuata (vedi 4), sono stati fissati dei parametri accuratamente selezionati, il cui senso logico è spiegato nella sezione 4.1.

La funzione sigmoideale è stata definita per tener conto di questo ragionamento sulla soglia. L'indice di copertura totale somma i singoli indici di copertura a cui è applicata la sigmoide, in modo che, durante il calcolo del gradiente (vedi 2), tutti i contributi degli agenti siano considerati.

Durante la simulazione, verranno generate delle posizioni iniziali degli agenti tali da avere una copertura leggermente inferiore al sufficiente nella situazione iniziale. L'obiettivo sarà quello di sviluppare un algoritmo di copertura dinamica (che verrà analizzato nella sezione 2) per creare le traiettorie degli agenti e indicare loro come muoversi, al fine di ottenere un *indice di copertura totale* finale significativamente maggiore rispetto a quello iniziale $E(\text{duration} - 1) > E(0)$.

È necessario garantire il più alto livello di copertura per il maggior tempo possibile.

Può succedere, come verrà analizzato in modo esaustivo nella sezione 4, che alcuni indici di coper-

tura singoli al tempo finale $E_{ij}(\text{duration} - 1)$ siano azzerati o prossimi allo zero. Questo comportamento non rappresenta un problema, ma è una conseguenza naturale dell'algoritmo di ascesa del gradiente descritto nella sezione 4.3. L'algoritmo mira a mantenere il grado di copertura più elevato possibile, massimizzando l'indice di copertura totale. Dato che il numero di agenti è inferiore alla metà del numero dei target, è normale che alcuni target possano non essere coperti nell'istante finale. Tuttavia, ciò consente di coprire molti altri target in modo ottimale, contribuendo così all'aumento dell'indice totale.

2 Algoritmi

Come anticipato nell'introduzione, l'obiettivo di questo progetto è sviluppare un algoritmo di copertura dinamico (**dynamic coverage algorithm**) per determinare le traiettorie degli agenti, al fine di massimizzare l'indice di copertura totale E .

Gli algoritmi proposti si basano tutti sul **metodo di ascesa del gradiente** come fondamento implementativo. Un algoritmo basato sull'ascesa del gradiente è una tecnica di ottimizzazione iterativa utilizzata per individuare il massimo di una funzione obiettivo. L'idea principale consiste nell'aggiornare iterativamente le variabili nella direzione del gradiente della funzione obiettivo rispetto a queste variabili, al fine di aumentare il valore della funzione (4).

Nelle sezioni seguenti verranno illustrati tre differenti tipi di algoritmi di copertura dinamica basati sull'ascesa del gradiente. Le versioni seconda e terza saranno estese per risolvere problemi di collisione tra gli agenti nell'istante finale.

Images/gradient_ascent_example.png

Figura 4: Ascesa del gradiente

2.1 Algoritmo di coverage V1: algoritmo di ascesa del gradiente classico

Nell'ambito del presente progetto di controllo della copertura dinamica, l'algoritmo di ascesa del gradiente viene impiegato per aggiornare le posizioni degli agenti mobili, come droni, determinando le loro traiettorie in modo da massimizzare una funzione obiettivo che rappresenta la copertura totale dell'area di interesse, ovvero l'indice di copertura totale $E(t)$. Gli agenti calcolano il gradiente della funzione di copertura rispetto alle loro posizioni e a quelle dei target al tempo corrente. Successivamente, aggiornano le loro traiettorie seguendo la direzione del gradiente, migliorando iterativamente la copertura complessiva dell'area.

Algorithm 1: V1: Algoritmo di coverage dinamico con ascesa del gradiente

Result: Massimizzare la copertura totale E

```

1 for ogni  $t$  do
2   for ogni agente  $i$  do
3     Decidere la direzione in cui muovere l'agente per massimizzare la copertura;
4     
$$\mu_i = \left. \frac{\partial E}{\partial p_i} \right|_{p_i=p_i(t), q_i=q_i(t)} (p_1, \dots, p_N, q_1, \dots, q_N)$$

5     L'agente si muove con passo di salita  $\epsilon$ ;
6      $p_i(t+1) = p_i(t) + \epsilon \mu_i(t)$ ;
7   end
8 end
```

In questo contesto, ϵ rappresenta il passo di salita. Questo parametro è cruciale per l'aggiornamento delle posizioni degli agenti mobili ed è direttamente responsabile della velocità con cui gli agenti si muovono lungo la traiettoria determinata dal gradiente della funzione obiettivo.

Come descritto in dettaglio nella sezione 1, è possibile che al termine dell'esecuzione dell'algoritmo alcuni indici di copertura individuali $E_{ij}(\text{duration} - 1)$ risultino pari a zero o prossimi allo zero. Questo fenomeno non rappresenta un'anomalia, ma un comportamento previsto dell'algoritmo di ascesa del gradiente, il cui obiettivo principale è la massimizzazione dell'indice di copertura totale. Di conseguenza, alcuni target potrebbero non essere coperti al termine della simulazione.

Un problema emergente in questa situazione è che gli agenti, per massimizzare la copertura, possono convergere verso posizioni finali molto vicine tra loro al tempo $t = \text{duration}$, aumentando il rischio di *collisione*. Questo è un problema ben noto nel contesto del coverage dinamico, dove la priorità alla copertura totale può portare a densità elevate di agenti in aree specifiche.

Per mitigare il rischio di collisione e migliorare la distribuzione spaziale degli agenti, sono state sviluppate due ulteriori versioni dell'algoritmo. Queste versioni estendono la versione base dell'algoritmo di ascesa del gradiente, introducendo meccanismi che favoriscono una separazione adeguata tra gli agenti, garantendo una copertura efficace senza incorrere in collisioni esagerate.

Nel contesto del controllo di copertura dinamica, il problema della collisione tra agenti è un aspetto cruciale da gestire. Quando più agenti, come droni, operano nello stesso spazio per monitorare una serie di punti di interesse, c'è un rischio significativo che le loro traiettorie si intersechino, portando a collisioni. Questo è particolarmente problematico alla fine dell'esecuzione dell'algoritmo di copertura, quando gli agenti tendono a convergere verso le aree con alta densità di target.

Per affrontare questo problema, diverse strategie di controllo dell'evitamento delle collisioni (*collision avoidance*) sono state proposte in letteratura. Alcuni approcci utilizzano tecniche di ottimizzazione che includono vincoli di distanza minima tra gli agenti, mentre altri implementano algoritmi di controllo distribuito che permettono agli agenti di coordinarsi per evitare le collisioni in tempo reale. Un esempio di queste tecniche è l'uso di funzioni potenziali che generano forze repulsive tra gli agenti quando si avvicinano troppo l'uno all'altro (5) (come fatto in V3: sezione 2.3).

2.2 Algoritmo V2: aggiunta di un moto Browniano

In questa seconda versione dell'algoritmo, le posizioni degli agenti vengono aggiornate non solo utilizzando il gradiente della funzione obiettivo, ma anche includendo una componente stocastica derivante dal **moto browniano**. Questo approccio introduce una variazione casuale nelle traiettorie degli agenti, migliorando la loro capacità di esplorazione e riducendo il rischio di convergenza in posizioni subottimali.

Algorithm 2: V2: Algoritmo di coverage dinamico, aggiunta moto di Browniano

Result: Massimizzare la copertura totale E

```

1 for ogni  $t$  do
2   Calcolo della componente Browniana  $\eta_i(t)$ 
3   for ogni agente  $i$  do
4     Decidere la direzione in cui muovere l'agente per massimizzare la copertura;
5
6     
$$\mu_i = \left. \frac{\partial E}{\partial p_i} \right|_{p_i=p_i(t), q_i=q_i(t)} (p_1, \dots, p_N, q_1, \dots, q_N)$$

7     L'agente si muove con passo di salita  $\epsilon$ ;
8      $p_i(t+1) = p_i(t) + \epsilon \mu_i(t) + \eta_i(t);$ 
9   end
10 end
```

Il moto browniano, noto anche come movimento aleatorio, è un modello matematico utilizzato per descrivere il comportamento di particelle che si muovono in modo casuale in un fluido. Questo fenomeno prende il nome dal botanico Robert Brown, che per primo osservò il movimento irregolare di particelle microscopiche in sospensione nel 1827. Il moto browniano è caratterizzato da una traiettoria irregolare e imprevedibile, dove ogni passo successivo è indipendente dal precedente e segue una distribuzione di probabilità specifica (6).

Il moto browniano può essere modellato matematicamente come un processo stocastico, spesso descritto da equazioni differenziali stocastiche. In particolare, il modello di Wiener è comunemente utilizzato per rappresentare il moto browniano. Questo modello considera la traiettoria di una particella come una somma di incrementi casuali, ciascuno distribuito normalmente con media zero e varianza proporzionale al tempo. La funzione di densità di probabilità per la posizione della particella a un tempo futuro è quindi una distribuzione gaussiana, il cui centro si sposta con il tempo e la cui larghezza aumenta proporzionalmente alla radice quadrata del tempo (6).

Nel contesto degli algoritmi di ottimizzazione e controllo, il moto browniano può essere utilizzato per introdurre una componente stocastica nei movimenti degli agenti. Questa componente stocastica permette una migliore esplorazione dello spazio delle soluzioni, contribuendo a evitare il problema della convergenza a soluzioni subottimali. In particolare, l'introduzione di una variazione casuale nei movimenti degli agenti consente loro di esplorare aree più ampie dello spazio di ricerca, aumentando così la probabilità di trovare soluzioni globali ottimali. Ad esempio, l'algoritmo di ottimizzazione basato sul moto browniano dei gas (GBMO) combina i principi fisici del moto browniano con strategie di ottimizzazione per migliorare la ricerca di soluzioni ottimali in spazi complessi (7). Altri studi dimostrano che il moto browniano può essere combinato con altri algoritmi di ottimizzazione, come l'algoritmo del volo della farfalla e l'ottimizzazione degli stormi di particelle, per migliorare ulteriormente le prestazioni e la convergenza (8), (9).

La componente browniana $\eta_i(t)$ è definita come:

$$\eta_i(t) = \begin{bmatrix} \text{gauss}_i(t) \cdot \cos(\theta_i(t)) \\ \text{gauss}_i(t) \cdot \sin(\theta_i(t)) \end{bmatrix} \quad (5)$$

dove:

$$\theta_i(t) \sim \mathcal{U}(0, 2\pi) \quad (6)$$

rappresenta la **direzione casuale**, distribuita uniformemente nell'intervallo $[0, 2\pi]$.

$$\text{gauss}_i(t) \sim \mathcal{N}(\mu, \sigma^2) \quad (7)$$

rappresenta **l'intensità del moto** browniano, distribuita secondo una distribuzione gaussiana con media μ e varianza σ^2 .

Questa componente aggiuntiva introduce una variazione casuale nel movimento degli agenti, permettendo una migliore esplorazione dello spazio e contribuendo a evitare che gli agenti si raggrupino in posizioni subottimali.

Images/Brownian_motion_components.png

Figura 5: Grafici del moto Browniano

La Figura 5 illustra diverse caratteristiche del moto browniano. Il grafico in alto a sinistra mostra la traiettoria in due dimensioni di una particella che segue un moto browniano, evidenziando la natura casuale e imprevedibile del percorso. I grafici in alto a destra e in basso a sinistra mostrano rispettivamente le componenti X e Y del moto browniano nel tempo, che evidenziano come queste componenti variano casualmente. Infine, l'istogramma in basso a destra rappresenta la distribuzione degli incrementi gaussiani, confermando che questi incrementi seguono una distribuzione normale con media zero.

2.3 Algoritmo V3: aggiunta di un potenziale repulsivo

In questa terza versione dell'algoritmo, le posizioni degli agenti vengono aggiornate non solo utilizzando il gradiente della funzione obiettivo, ma anche includendo una componente di **potenziale repulsivo**. Questo approccio introduce una forza repulsiva tra gli agenti quando la distanza tra di loro è inferiore a una certa soglia δ . L'obiettivo principale di questa componente è evitare collisioni tra gli agenti durante la loro operatività, migliorando la sicurezza e l'efficacia del controllo della copertura.

Algorithm 3: V3: Algoritmo di coverage dinamico, aggiunta di un potenziale repulsivo

Result: Massimizzare la copertura totale E

```

1 for ogni  $t$  do
2   for ogni agente  $i$  do
3     Calcolo del potenziale repulsivo
4     Decidere la direzione in cui muovere l'agente per massimizzare la copertura;
5
6     
$$\mu_i = \left. \frac{\partial E}{\partial p_i} \right|_{p_i=p_i(t), q_i=q_i(t)} (p_1, \dots, p_N, q_1, \dots, q_N)$$

7     L'agente si muove con passo di salita  $\epsilon$ ;
8      $p_i(t+1) = p_i(t) + \epsilon \mu_i(t) + \eta$ ;
9   end
10 end

```

La **forza repulsiva totale** η è la somma delle componenti repulsive calcolate per tutti gli agenti k diversi da i :

$$\eta = \sum_{k \neq i} \eta_i^k(t) \quad (8)$$

dove la componente repulsiva $\eta_i^k(t)$ è calcolata come segue:

$$\eta_i^k(t) = \begin{cases} \left[\frac{p_i(t) - p_k(t)}{d_{ik}(t)} \right] \cdot \left[\frac{\delta - d_{ik}(t)}{d_{ik}(t)} \right] & \text{se } d_{ik}(t) \leq \delta \\ 0 & \text{se } d_{ik}(t) > \delta \end{cases} \quad (9)$$

dove $d_{ik}(t) = \|p_i(t) - p_k(t)\|$ rappresenta la distanza tra l'agente i e l'agente k al tempo t .

Se la distanza è inferiore alla soglia δ , viene calcolata una forza repulsiva. Il primo termine **in blu** è il versore che indica *la direzione di allontanamento* dall'agente k . Il secondo termine **in arancione** rappresenta *l'intensità della forza repulsiva*, che aumenta inversamente proporzionale alla distanza tra gli agenti: più i e k sono vicini, maggiore è la forza repulsiva.

Questo fattore è critico per garantire che la forza repulsiva sia efficace solo quando necessario, senza interferire con il compito principale di massimizzare la copertura.

Viene riportato il grafico dell' **intensità della forza repulsiva** che mostra come l'intensità della forza repulsiva decresca all'aumentare della distanza tra gli agenti i e k . Quando la distanza è inferiore

a una certa soglia δ , la forza repulsiva è alta, mentre diminuisce rapidamente man mano che la distanza aumenta, tendendo a zero quando la distanza è maggiore di δ .



Figura 6: Intensità del potenziale repulsivo

Il potenziale repulsivo è una tecnica ampiamente utilizzata nel campo della robotica e del controllo multi-agente per prevenire le collisioni. La forza repulsiva cresce man mano che gli agenti si avvicinano tra loro, inducendo un movimento che li spinge a distanziarsi. Questo metodo garantisce che gli agenti mantengano una distanza di sicurezza reciproca, riducendo il rischio di collisioni durante il movimento.

Secondo Hao et al., il potenziale repulsivo è fondamentale per il controllo di formazione e la manutenzione della connettività nei sistemi multi-agente. La forza repulsiva viene calcolata in base alla distanza tra gli agenti, aumentando esponenzialmente quando la distanza scende sotto una

soglia di sicurezza, generando un'azione di repulsione che allontana gli agenti l'uno dall'altro (10). Questo meccanismo è particolarmente utile nel contesto del coverage dinamico, dove gli agenti devono cooperare per coprire un'area di interesse senza collidere tra loro.

In un altro studio, Jiang et al. descrivono come l'utilizzo di potenziali repulsivi possa migliorare l'efficacia della copertura dinamica, specialmente in scenari complessi con molti agenti che operano in prossimità. I potenziali repulsivi permettono agli agenti di distribuire uniformemente lo spazio di copertura, ottimizzando l'efficienza e riducendo i rischi di collisione (11).

Inoltre, Nagy et al. hanno condotto uno studio sul comportamento dei sistemi multi-agente mobili durante la costruzione di campi di potenziale, dimostrando che le forze repulsive aiutano a mantenere gli agenti separati e ad evitare collisioni (12). L'integrazione del controllo reattivo e telerobotico nei sistemi robotici multi-agente è stata esplorata da Arkin e Ali, evidenziando come le forze repulsive tra robot permettano una convergenza sicura e coordinata (13). Infine, Mabrouk e McInnes hanno introdotto un modello per simulare il comportamento emergente dei robot a sciame, utilizzando campi di potenziale basati su forze attrattive e repulsive (14).

Questo approccio assicura che ogni agente consideri la presenza degli altri agenti nel suo ambiente e regoli il proprio movimento di conseguenza, *evitando collisioni* e mantenendo una copertura efficace dell'area di interesse.

3 Implementazione degli algoritmi

L'intero progetto di tesi è stato sviluppato utilizzando il linguaggio di programmazione Python. In questa sezione, verranno illustrati SOLO I PUNTI CRUCIALI DELLA LOGICA DEL PROGRAMMA, partendo dalle operazioni preliminari fino ad arrivare alla reale implementazione degli algoritmi di coverage discussi nella sezione 2.

Per l'analisi del codice completo è possibile fare riferimento a ([Source code](#)).

Il codice è strutturato come mostrato nelle figure 7 e 8.



Figura 7: Organizzazione del codice



Figura 8: Zoom nelle cartelle degli algoritmi

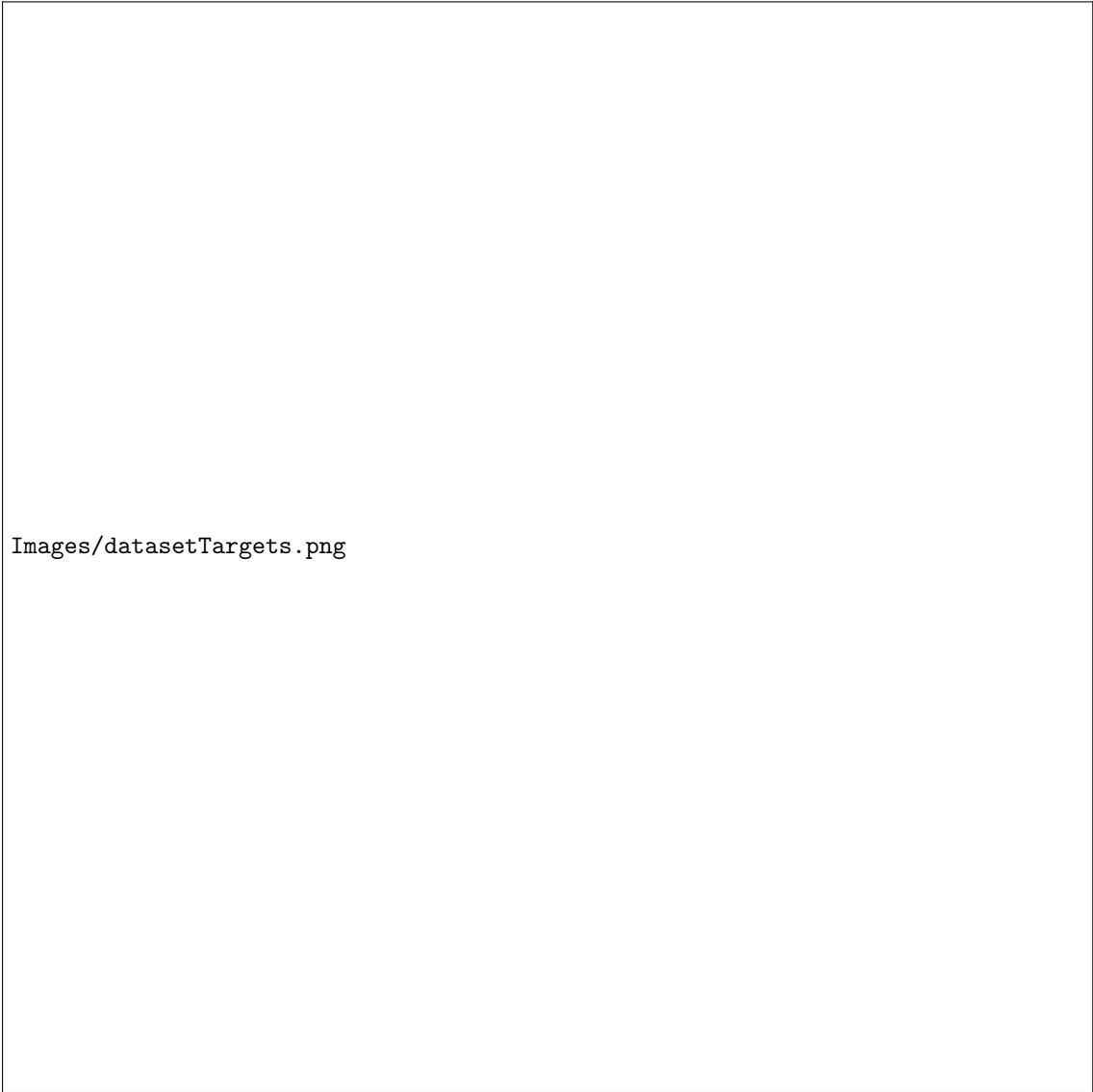
3.1 Operazioni preliminari

L'unico dato in input di questo progetto è un dataset, salvato in un file *.pickle*. Come illustrato in figura 9, il dataset contiene le traiettorie dei target. Una volta caricato, il dataset (50 traiettorie in una regione di 800 x 800) si presenta nella seguente struttura:

- `np.shape(dataset[i][0])=(351, 4)`

- `np.shape(dataset[i][1])=(351, 2)`

dove *i* è l'indice della traiettoria e 0,1 indica stato vero (0) con 4 componenti (posizione x, posizione y, velocità x, velocità y) e misure (1) con 2 componenti della posizione, in tutto ciò *l'intervallo di campionamento è di 1 secondo*. Quindi il dataset è una lista di tuple, ognuna di esse contiene due array numpy, uno per rappresentare lo stato vero della traiettoria e l'altro per le **misurazioni** (che rappresentano i dati effettivi delle coordinate della traiettoria considerando lo stato vero + un rumore di misura additivo).



Images/datasetTargets.png

Figura 9: Dataset traiettorie dei target

Per simulare il progetto, ad ogni esecuzione del programma vengono selezionate casualmente 10 traiettorie (*numTargets*) della durata di 150 secondi (*duration*). Maggiori dettagli sul settaggio dei parametri sono disponibili nella sezione 4. Di seguito è riportato il cuore della funzione responsabile della creazione del dataset di traiettorie dei target utilizzate. È importante notare che viene estratta solo la parte delle **misurazioni**, che è quella di nostro interesse. Ovviamente prima di chiamare la funzione, nel main, il dataset è stato estratto dal file pickle (vedi (15)).

Snippet 3.1: Metodo "buildTheSet()" di *createTheSetOfTargets*

```
1 def buildTheSet(numTrajectories: int, inputDataset: list, duration:
    int):
```



```

2     selectedTrajectories = []
3     try:
4         for _ in range(numTrajectories):
5             random_trajectory_index = np.random.choice(len(
6                 inputDataset))
7             # Ottieni la traiettoria corrispondente all'indice
8             # casuale
9             random_trajectory = inputDataset[random_trajectory_index]
10            # Tronca la traiettoria ai primi "duration" secondi
11            truncated_real_state = random_trajectory[0][:duration]
12            truncated_measurement = random_trajectory[1][:duration]
13            # Aggiungi la traiettoria troncata alla lista delle
14            # traiettorie selezionate
15            selectedTrajectories.append((truncated_real_state,
16                truncated_measurement))
17    except Exception as e:
18        print("Errore durante la creazione dello scenario:", e)
19
20    return selectedTrajectories

```

Snippet 3.2: Estrazione delle misure nel *main*

```

1 # Estrarre solo la parte truncated_measurement
2 targets = [truncated_measurement for _, truncated_measurement in
3             createdDatasetOfTargets]
4 targetTrajectories = np.array(targets)

```

La struttura dell'array *targetTrajectories* sarà dunque $[\text{NUMTARGETS}] \times [\text{DURATION}] \times [2]$ dove 2 rappresenta le due coordinate x e y.

Successivamente, è necessario andare a creare delle posizioni iniziali degli agenti, dei quali, per adesso, non è presente nessuna informazione. Attraverso una verifica, eseguita creando e visualizzando il grafico delle posizioni iniziali dei target, è stato rilevato che queste sono tutte comprese nell'area $[0,200] \times [0,200]$ (*initialAreaSize*). Questa informazione consente di posizionare gli agenti inizialmente in modo uniforme all'interno di questa area. Nel progetto sono stati utilizzati 10 target e 4 agenti (*numAgents*), ma l'approccio è valido anche per un numero diverso di agenti (aspetto migliorabile, come verrà discusso nelle conclusioni).

Snippet 3.3: Metodo "generateInitialAgentPositions()" di *createTheSetOfAgents*

```

1 if numAgents == 4:
2     initialAgentPositions = [(50, 50), (50, 150), (150, 50),
3                             (150, 150)]

```

```

3     else:
4         # Calcola la dimensione della griglia
5         grid_size = int(np.sqrt(numAgents))
6         if grid_size * grid_size < numAgents:
7             grid_size += 1
8         # Calcola le distanze tra gli agenti per coprire l'area 200
           x200
9         x_spacing = (initialAreaSize - 100) / (grid_size - 1)
10        y_spacing = (initialAreaSize - 100) / (grid_size - 1)
11        initialAgentPositions = []
12        # Genera le posizioni degli agenti distribuendoli
           uniformemente
13        for i in range(grid_size):
14            for j in range(grid_size):
15                if len(initialAgentPositions) < numAgents:
16                    x = 50 + j * x_spacing
17                    y = 50 + i * y_spacing
18                    initialAgentPositions.append((x, y))
19        return np.array(initialAgentPositions, dtype=float)

```

La struttura dell'array *initialAgentsPositions* sarà dunque $[\text{NUMAGENTS}] \times 2$.

Come ultima operazione "preliminare", ma non meno importante, è necessario calcolare gli indici di copertura all'istante iniziale $E_j(0)$ per tutti i target j e anche l'indice di copertura totale iniziale $E(0)$.

Questi calcoli sono fondamentali per effettuare i confronti necessari a dimostrare il corretto funzionamento del progetto, ovvero provare che l'indice di copertura totale finale $E(\text{duration} - 1)$ è significativamente maggiore rispetto a quello iniziale.

Il codice per questi calcoli è contenuto nel modulo *initialCoverageIndices.py* e i vari metodi implementati rispecchiano precisamente ciò che è stato descritto nella formulazione del problema (1.1) e nelle soluzioni proposte (1.2).

3.2 Descrizione del codice (e tabelle con pseudocodice)

Sono state descritte tutte le operazioni preliminari necessarie per impostare il progetto, tra cui l'estrazione delle traiettorie dei target, la creazione delle posizioni iniziali degli agenti e il calcolo degli indici di copertura iniziali. Questi passaggi preparatori sono fondamentali per l'implementazione degli algoritmi di copertura dinamica descritti nella sezione 2.

In questa sezione, verrà illustrata l'effettiva implementazione degli algoritmi di coverage, discutendo gli aspetti più significativi.

3.2.1 Versione 1

In questa sottosezione viene illustrato il **cuore del progetto**: l'algoritmo di copertura dinamica basato sull'ascesa del gradiente. L'implementazione segue fedelmente lo pseudocodice descritto in sezione 2.1, come mostrato nel codice sottostante.

Snippet 3.4: Metodo "coverageAlgorithmV1()" di *v1.coverageAlgorithmV1*

```
1 def coverageAlgorithmV1(targetsTrajectories: list,  
    agentsInitialPosition: list, r, mp, lb, h, NUMAGENTS, NUMSECONDS,  
    EPSILON):  
2     agentsInitialPositions = np.array(agentsInitialPosition)  
3  
4     agentsTrajectories = np.zeros((NUMSECONDS, NUMAGENTS, 2))  
5  
6     agentsTrajectories[0] = agentsInitialPositions  
7  
8     for t in range(NUMSECONDS-1):  
9         gradients_t = gradientOfCoverageIndex(targetsTrajectories,  
            agentsTrajectories[t], t, r, mp, lb, h)  
10  
11         for i in range(NUMAGENTS):  
12             agentsTrajectories[t+1][i][0] = (agentsTrajectories[t][i]  
                [0]) + (EPSILON*gradients_t[i][0])  
13             agentsTrajectories[t+1][i][1] = (agentsTrajectories[t][i]  
                [1]) + (EPSILON*gradients_t[i][1])  
14     return agentsTrajectories
```

Come si può notare, all'algoritmo vengono passati in ingresso vari parametri fondamentali, i quali verranno descritti meglio in sezione 4. Successivamente, viene inizializzato l'array numpy che conterrà le traiettorie degli agenti, con struttura $[DURATION] \times [NUMAGENTS] \times [2]$.

Ad ogni istante di tempo, corrispondente a un secondo (che rappresenta l'intervallo di campionamento), viene calcolato il gradiente dell'indice di copertura totale $E(t)$ utilizzando il metodo *gradientOfCoverageIndex()*, che calcola anche i singoli indici di copertura. Successivamente, vengono predette e salvate le future posizioni degli agenti.

In questo modo, alla fine dell'algoritmo, l'array numpy *agentsTrajectories* conterrà le traiettorie complete degli agenti, che seguono i target al fine di massimizzare l'indice di copertura totale.

E' interessante approfondire come è stato implementato il metodo *gradientOfCoverageIndex()*.

Inizialmente veniva utilizzato il metodo alle differenze finite. Il metodo delle differenze finite è una tecnica numerica utilizzata per approssimare le derivate di una funzione, particolarmente utile quando la derivata analitica è difficilmente calcolabile o si lavora con dati discreti. Per calcolare il gradiente di una funzione $f(x)$ in un punto x , si utilizza una piccola perturbazione h . Una formula comune è la differenza finita in avanti:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (10)$$

Dove:

- $f'(x)$ è la derivata approssimata della funzione f nel punto x
- h è un piccolo incremento, chiamato passo, che rappresenta la distanza tra i punti utilizzati per calcolare la differenza finita. Il valore di h deve essere sufficientemente piccolo per garantire una buona approssimazione, ma non troppo piccolo da causare problemi numerici, come la cancellazione numerica (ed errori numerici)

Per funzioni in più variabili, il gradiente ∇f può essere calcolato utilizzando le differenze finite per ogni componente. Ad esempio, per una funzione $f(x_1, x_2, \dots, x_n)$, il gradiente è un vettore delle derivate parziali:

$$\nabla f \approx \left(\frac{f(x_1+h, x_2, \dots, x_n) - f(x_1, x_2, \dots, x_n)}{h}, \dots, \frac{f(x_1, \dots, x_n+h) - f(x_1, \dots, x_n)}{h} \right) \quad (11)$$

Questa tecnica è ben documentata e utilizzata in vari contesti ingegneristici e scientifici (16; 17).

Il metodo delle differenze finite, purtroppo, si è rivelato insufficiente per il nostro scopo, in quanto ha introdotto numerose problematiche. In particolare, **il calcolo del gradiente** ha mostrato oscillazioni troppo brusche e anomalie significative. Per superare queste difficoltà, abbiamo adottato *PyTorch*, una libreria open-source per il machine learning.

Per il codice della funzione e ulteriori dettagli, si rimanda il lettore alla sezione 3.3.4.

3.2.2 Versione 2

Come descritto ampiamente nella sezione 2.1, per mitigare il rischio di collisione e migliorare la distribuzione spaziale degli agenti, nella seconda versione dell'algoritmo è stato introdotto un moto Browniano. Di seguito è riportata l'implementazione, dove si notano le differenze e le aggiunte rispetto all'algoritmo precedente.

Snippet 3.5: Metodo "coverageAlgorithmV2()" di *v2.coverageAlgorithmV2*

```
1 def coverageAlgorithmV1(targetsTrajectories: list,
2   agentsInitialPosition: list, r, mp, lb, h, NUMAGENTS, NUMSECONDS,
3   EPSILON):
4   agentsInitialPositions = np.array(agentsInitialPosition)
5
6   agentsTrajectories = np.zeros((NUMSECONDS, NUMAGENTS, 2))
7
8   agentsTrajectories[0] = agentsInitialPositions
9
10  #Creazione delle traiettorie del moto browniano per la durata
11  #specificata
12  m_i_t = creationOfBrownianMotionTrajectories(NUMSECONDS)
13
14  for t in range(NUMSECONDS-1):
15    gradients_t = gradientOfCoverageIndex(targetsTrajectories,
16      agentsTrajectories[t], t, r, mp, lb, h)
17
18    for i in range(NUMAGENTS):
19      agentsTrajectories[t+1][i][0] = (agentsTrajectories[t][i]
20        [0]) + (EPSILON*gradients_t[i][0]) + m_i_t[t][0]
21      agentsTrajectories[t+1][i][1] = (agentsTrajectories[t][i]
22        [1]) + (EPSILON*gradients_t[i][1]) + m_i_t[t][1]
23  return agentsTrajectories
```

Come descritto in sezione 2.2, in questo algoritmo vengono introdotte delle variazioni casuali nelle traiettorie degli agenti secondo un moto Browniano

Snippet 3.6: Metodo "brownianMotionDef()" di *v2.brownianMotion*

```
1 def brownianMotionDef(t, mu, sigma, n):
2   theta_i = uniformDistribution(n)
3   gauss_i = normalDistribution(mu, sigma, n)
4   x = gauss_i * np.cos(theta_i)
5   y = gauss_i * np.sin(theta_i)
6   return np.vstack((x, y)).T
```

Il metodo *brownianMotionDef* calcola il moto browniano per un insieme di punti temporali. Questo metodo genera una sequenza di vettori di movimento (coordinate x e y) utilizzando distribuzioni gaussiana e uniforme per determinare le intensità e le direzioni dei movimenti.

- **Angoli direzionali casuali:** Viene chiamata la funzione *uniformDistribution(n)*, che restituisce n campioni dalla distribuzione uniforme $U(0, 2\pi)$. Questi campioni rappresentano gli angoli direzionali casuali del movimento browniano.

- **Intensità del movimento:** Viene chiamata la funzione *normalDistribution(mu, sigma, n)*, che restituisce n campioni dalla distribuzione gaussiana $N(\mu, \sigma^2)$. Questi campioni rappresentano le intensità (lunghezze) casuali del movimento.
- **Coordinate x e y:** Le coordinate x e y dei vettori di movimento vengono calcolate utilizzando le funzioni trigonometriche $\cos$ e $\sin$ per determinare rispettivamente le componenti orizzontali e verticali del vettore.
- **Matrice finale:** Le coordinate x e y vengono combinate in una matrice 2D utilizzando *np.vstack((x, y))*, che impila gli array x e y verticalmente. La trasposizione della matrice $(.T)$ assicura che ogni riga rappresenti un vettore $[x, y]$ per ogni punto temporale. Il risultato è una matrice con forma $(n, 2)$, dove n è il numero di campioni.

Il metodo restituisce una matrice $[\text{NUMAGENTS}] \times [2]$ con i vettori di movimento per ogni punto temporale, consentendo la simulazione del moto browniano in un piano bidimensionale.

3.2.3 Versione 3

Un'altra variante dell'algoritmo base, descritta nella sezione 2.3, prevede l'aggiornamento delle posizioni degli agenti utilizzando non solo il gradiente della funzione obiettivo, ma anche una componente di **potenziale repulsivo**. Questa componente aggiuntiva aiuta a prevenire le collisioni tra gli agenti, migliorando la distribuzione spaziale e l'efficacia complessiva del controllo di copertura.

Snippet 3.7: Metodo "coverageAlgorithmV3()" di *v3.coverageAlgorithmV3*

```

1 def coverageAlgorithmV1(targetsTrajectories: list,
   agentsInitialPosition: list, r, mp, lb, h, NUMAGENTS, NUMSECONDS,
   EPSILON, DELTA):
2     #... codice iniziale uguale alle altre versioni
3     for t in range(NUMSECONDS-1):
4         gradients_t = gradientOfCoverageIndex(targetsTrajectories,
           agentsTrajectories[t], t, r, mp, lb, h)
5         for i in range(NUMAGENTS):
6             potRep = calculateRepulsivePotential(i,
                   agentsTrajectories[t], DELTA)
7             agentsTrajectories[t+1][i][0] = (agentsTrajectories[t][i]
                   ][0]) + (EPSILON*gradients_t[i][0]) + potRep[0]
8             agentsTrajectories[t+1][i][1] = (agentsTrajectories[t][i]
                   ][1]) + (EPSILON*gradients_t[i][1]) + potRep[1]
9     return agentsTrajectories

```

Ogni secondo (intervallo di campionamento), per ciascun agente i , viene calcolato un potenziale repulsivo che contribuisce all'aggiornamento delle loro posizioni. Come descritto nella sezione 2.3, il potenziale repulsivo è determinato nel seguente modo:

Snippet 3.8: Metodo "calculateRepulsivePotential()" di *repulsivePotential*

```
1 def calculateRepulsivePotential(i, agentsPositions, delta):
2     potRep = np.zeros_like(agentsPositions[i])
3
4     for k in range(len(agentsPositions)):
5         if k != i:
6             diff = agentsPositions[i] - agentsPositions[k]
7             d_ik = np.sqrt(diff[0]**2 + diff[1]**2)
8
9             if d_ik <= delta:
10                 eta_ik = ((agentsPositions[i] - agentsPositions[k]) /
11                           d_ik) * (delta - d_ik) / d_ik
12             else:
13                 eta_ik = np.zeros_like(agentsPositions[i])
14             potRep += eta_ik
15
16     return potRep
```

Il metodo *calculateRepulsivePotential()* calcola il potenziale repulsivo per un agente specifico all'interno di un gruppo di agenti per evitare collisioni. La logica del codice segue perfettamente quanto esposto nell'equazione 9.

Quando il valore ritornato dal metodo è negativo, indica che l'agente deve allontanarsi dagli altri agenti vicini, un comportamento desiderato per evitare collisioni. Quando il potenziale ha componenti negative, la forza repulsiva spinge l'agente nella direzione opposta, aumentando la distanza tra gli agenti.

3.3 Librerie Python utilizzate

Di seguito vengono elencate ed illustrate le librerie Python (degne di nota) utilizzate per raccogliere, elaborare ed analizzare i dati necessari alla scrittura degli algoritmi di copertura. L'implementazione è stata realizzata utilizzando Python nella versione 3.11.3.

3.3.1 NumPy

NumPy è una libreria fondamentale per il calcolo scientifico in Python, che fornisce supporto per array e matrici multidimensionali, oltre a numerose funzioni matematiche (18). Grazie alla sua implementazione in C, NumPy è altamente efficiente e viene utilizzata in una vasta gamma di applicazioni scientifiche e ingegneristiche.

Nel progetto di tesi, NumPy è stato utilizzato per diverse operazioni chiave:

- **Gestione delle traiettorie:**
 - Caricamento e manipolazione del dataset delle traiettorie, organizzate in array multidimensionali.
 - *File di riferimento: `createTheSetOfTargets.py`.*
- **Inizializzazione della posizione degli agenti:**
 - Generazione delle posizioni iniziali degli agenti all'interno di un'area specificata utilizzando `np.random.uniform`.
 - *File di riferimento: `createTheSetOfAgents.py`.*
- **Calcolo degli indici di copertura:**
 - Calcolo efficiente degli indici di copertura iniziali tramite operazioni vettoriali e matriciali.
 - *File di riferimento: `initialCoverageIndices.py`.*
- **Implementazione degli algoritmi di coverage:**
 - Aggiornamento delle posizioni degli agenti, calcolo dei gradienti e delle componenti browniane negli algoritmi di coverage.
 - *File di riferimento: `coverageAlgorithmV1/2/3.py`*
- **Operazioni matematiche avanzate:**
 - Calcolo delle distanze, interpolazioni e altre operazioni matematiche avanzate necessarie per gli algoritmi implementati.

NumPy è stato cruciale per gestire array, calcolare distanze e gradienti, dimostrando la sua importanza e versatilità nel progetto di tesi.

3.3.2 Pickle

Pickle è una libreria standard di Python che permette la serializzazione e deserializzazione di oggetti Python. La serializzazione è il processo di conversione di un oggetto in una sequenza di byte per memorizzarlo in un file o trasferirlo attraverso una rete. La deserializzazione è l'operazione inversa, ossia convertire la sequenza di byte in un oggetto Python. Nel progetto, 'pickle' è stato utilizzato per:

- **Caricamento delle traiettorie dei target:**
 - Utilizzato per caricare un dataset di traiettorie salvato in un file `.pickle`.
 - *File di riferimento: `createTheSetOfTargets.py`.*

3.3.3 Matplotlib

Matplotlib è una libreria di plotting per Python che fornisce un framework flessibile e potente per la visualizzazione di dati. Consente di creare grafici statici, animati e interattivi (20). Nel progetto, Matplotlib è stata utilizzata per:

- **Visualizzazione delle traiettorie:**
 - Tracciare e visualizzare le traiettorie dei target e degli agenti durante la simulazione.
 - *File di riferimento: `coverageAlgorithmV1/2/3.py`.*
- **Grafici degli indici di copertura:**
 - Creare grafici che mostrano l'evoluzione degli indici di copertura nel tempo, facilitando l'analisi dei risultati.
 - *File di riferimento: `coverageIndices.py`.*
- **Grafici utili alla relazione:**
 - Generare tutti i grafici illustrati nella relazione per migliorarne la comprensione e l'impatto visivo.
 - *File di riferimento: `CodiciGenerazioneGrafici`.*

3.3.4 Pytorch

PyTorch è una libreria open-source per il machine learning sviluppata da Facebook's AI Research lab. È utilizzata principalmente per applicazioni di deep learning e reti neurali, grazie alla sua flessibilità, velocità di esecuzione e supporto per il calcolo su GPU. PyTorch offre un'interfaccia intuitiva per la creazione di modelli complessi e fornisce potenti strumenti per il calcolo automatico del gradiente, rendendola una scelta ideale per la ricerca e lo sviluppo in ambito di intelligenza artificiale.

PyTorch è particolarmente potente per il *calcolo del gradiente*, che è essenziale per il training dei modelli di machine learning e deep learning. Ecco come PyTorch facilita questo processo:

- **Autograd:** PyTorch include una libreria di differenziazione automatica chiamata Autograd, che traccia tutte le operazioni che coinvolgono i tensori e costruisce dinamicamente un grafo computazionale. Ogni nodo del grafo rappresenta una variabile tensoriale e i bordi rappresentano le operazioni che producono nuovi tensori.
- **Backward Propagation:** Una volta costruito il grafo computazionale, PyTorch può calcolare automaticamente i gradienti rispetto a qualsiasi variabile utilizzando il metodo di backpropagation. Chiamando `.backward()` su un tensore, PyTorch calcola il gradiente di quel tensore rispetto a tutti i tensori che hanno contribuito alla sua creazione.
- **Efficienza:** PyTorch è ottimizzato per sfruttare le capacità delle GPU, rendendo il calcolo dei gradienti estremamente efficiente, cruciale per l'allenamento di modelli di deep learning su grandi dataset.
- **Supporto per le Funzioni Custom:** Gli utenti possono definire nuove funzioni custom e PyTorch può automaticamente calcolarne i gradienti, utile per implementare nuovi algoritmi di machine learning che richiedono operazioni personalizzate.
- **Interfaccia Intuitiva:** PyTorch offre un'interfaccia intuitiva e Pythonica che rende semplice scrivere e comprendere i modelli, facilitando il processo di calcolo del gradiente e il debug.

In sintesi, PyTorch semplifica il calcolo del gradiente attraverso la sua libreria Autograd, permettendo una definizione dinamica e efficiente dei grafi computazionali, supportata da accelerazione hardware. Questa funzionalità è fondamentale per l'allenamento e l'ottimizzazione dei modelli di machine learning e deep learning (vedi (21)).

Nel contesto del progetto, PyTorch è stato utilizzato principalmente per calcolare i gradienti, migliorando l'efficienza e la precisione rispetto al metodo delle differenze finite. Di seguito vengono descritti i vari usi specifici di PyTorch nel codice.

- **Calcolo Automatico del Gradiente:**
 - PyTorch è stato utilizzato per calcolare i gradienti delle funzioni obiettivo in modo efficiente e preciso.
 - La funzione `gradientOfCoverageIndex` utilizza PyTorch per derivare i gradienti, migliorando la precisione e riducendo le oscillazioni rispetto alle differenze finite.
 - *File di riferimento:* `gradient.py`.
- **Definizione e Manipolazione dei Tensors:**
 - PyTorch permette la creazione e manipolazione di tensors, che sono l'equivalente dei arrays in NumPy, ma ottimizzati per il calcolo su GPU.
 - Utilizzato per rappresentare e gestire le traiettorie e le posizioni degli agenti e dei target durante l'esecuzione degli algoritmi.
 - *File di riferimento:* `coverageIndices.py`.

- **Integrazione con Altri Strumenti:**

- PyTorch si integra facilmente con altre librerie utilizzate nel progetto, come NumPy e Matplotlib, permettendo un workflow fluido per l'analisi e la visualizzazione dei dati.
- Questa integrazione ha facilitato la transizione dei dati tra le varie fasi di calcolo, visualizzazione e analisi.

PyTorch si è rivelato fondamentale per l'implementazione efficiente e precisa degli algoritmi di copertura dinamica, dimostrando la sua versatilità e potenza nel contesto del machine learning e dell'intelligenza artificiale.

Vediamo come Pytorch è stata utilizzata nel modulo *gradient.py* per calcolare il gradiente dell'indice di copertura totale:

Snippet 3.9: Metodo "gradientOfCoverageIndex()" di *gradient*

```
1 def gradientOfCoverageIndex(targets, agentsPosition, t, r, mp, lb, h)
2     :
3     # Conversione della posizione degli agenti in un tensore PyTorch
4     # e abilitazione del calcolo dei gradienti
5     agentsPosition_torch = torch.tensor(agentsPosition, requires_grad_
6     =True, dtype=torch.float32)
7     # Calcolo degli indici di copertura dei target
8     coverageIndices = calculateCoverageIndices(targets,
9     agentsPosition_torch, t, r, mp)
10    # Calcolo dell'indice di copertura totale
11    totalCoverageIndex_t = calculateTotalCoverageIndex(
12    coverageIndices, lb)
13    # Assicurarsi che l'indice di copertura totale richieda gradienti
14    totalCoverageIndex_t = totalCoverageIndex_t.requires_grad_(True)
15    # Esecuzione della backpropagation per calcolare i gradienti
16    totalCoverageIndex_t.backward()
17    # Verifica se i gradienti sono stati calcolati correttamente
18    if agentsPosition_torch.grad is None:
19        raise RuntimeError("Gradienti calcolati erroneamente")
20    # Detach i gradienti dal grafo computazionale di PyTorch e
21    # convertirli in un array NumPy
22    gradients_t = agentsPosition_torch.grad.detach().numpy()
23    return gradients_t
```

4 Simulazione e discussione dei risultati

In questa sezione vengono presentate le simulazioni effettuate per valutare l'efficacia degli algoritmi di copertura dinamica sviluppati nel corso di questo lavoro. **L'obiettivo principale** delle simulazioni è verificare la capacità degli algoritmi di massimizzare l'indice di copertura totale $E(t)$ e garantire una distribuzione ottimale degli agenti nello spazio di interesse.

4.1 Setting dei parametri iniziali

Il codice implementato e descritto dettagliatamente nella sezione 3.2 è stato progettato per massimizzare l'indice di copertura totale, indipendentemente dalle traiettorie casuali selezionate nel dataset fornito. Tuttavia, per la redazione di questa sezione della relazione, sono state fissate dieci traiettorie tra le 50 disponibili nel dataset, estratte in modo casuale durante un'esecuzione del programma. Le traiettorie sono: 5-esima, 40-esima, 26-esima, 45-esima, 49-esima, 21-esima, 0-esima, 6-esima, 15-esima, 43-esima.

Facendo riferimento a quanto già accennato nella sezione 1.2, è importante ricordare che i parametri del progetto sono strettamente interdipendenti, a causa della relazione tra Mp , la funzione di misurazione $E_{ij}(p_i(t), q_j(t))$ e i vari indici di copertura. Pertanto, è necessario scegliere con attenzione i vari parametri per garantire un senso logico e un risultato corretto nella simulazione. Di seguito verranno riportati tutti i parametri fissati per la simulazione, con annesse spiegazioni per la scelta di ciascuno:

- **numTargets & numAgents:** Sono stati scelti 10 targets e 4 agenti. Questa configurazione è stata selezionata per mantenere il numero di agenti inferiore alla metà dei targets, evitando un rapporto 1:1 che renderebbe il progetto troppo semplice e banale. Inoltre, questa scelta è in linea con i parametri utilizzati nell'articolo scientifico (2), fornendo un contesto di riferimento consolidato per la simulazione.
- **initialAreaSize:** Questo parametro fornisce alla funzione *generateInitialAgentPositions()* le informazioni necessarie per generare uniformemente le posizioni iniziali degli agenti. Come valore è stato preso '200'; il motivo lo si può notare dalla figura 9.
- **duration:** La durata delle traiettorie dei target è stata fissata a 150 secondi, con un intervallo di campionamento di 1 secondo. Ciò implica che lo scenario della simulazione comprende 150 TIME STEPS. Questo valore è stato scelto per diverse ragioni: è leggermente superiore a quello utilizzato nell'articolo scientifico di riferimento (120 secondi), e rappresenta un compromesso ottimale tra l'uso delle risorse computazionali e la convergenza dell'algoritmo.
- **r:** Questo parametro rappresenta il raggio di copertura di ogni agente, fissato a 150. Questa scelta è stata effettuata proporzionando la grandezza dello scenario e il raggio di copertura utilizzati nell'articolo scientifico di riferimento alla dimensione iniziale del nostro scenario (*initialAreaSize*).
- **Mp:** La qualità di rilevamento di picco (*peak sensing quality*) è uno dei **parametri fondamentali** da stabilire, poiché influisce sull'intera logica del progetto. La funzione di misurazione varierà nell'intervallo $[0, Mp]$, mentre gli indici di copertura dei target j saranno

compresi tra 0 e $Mp \cdot \text{numAgents}$. Mp è stato impostato a 1, come in (2). Considerando questi parametri numerici, possiamo affermare che la funzione di misurazione varia tra 0 e 1, mentre gli indici di copertura variano tra 0 e 4. Impostare la soglia E^* a 2 implica che, nel complesso, desideriamo che almeno 2 agenti su 4 coprano ciascun target. Per raggiungere il valore 2, è possibile che 2 agenti siano esattamente sopra i target e abbiano quindi copertura massima, oppure che la somma delle funzioni di misurazione complessive sia pari a 2 (ogni agente copre in minima parte quel target, sommando le coperture si raggiunge la soglia).

- **lowerBoundIndex:** *Corrisponde alla soglia E^** descritta nella sezione 1.2. Questo parametro è stato impostato a 2. Ma cosa significa? Con i dati analizzati fino ad ora, sappiamo che $E_j(t)$ varia tra 0 e 4, e ogni agente contribuisce con un massimo di 1 (come descritto nell'equazione 2). Impostare E^* a 2 significa che vogliamo ottenere una copertura equivalente a quella di due agenti perfettamente posizionati sopra il target, o che la somma delle funzioni di misurazione complessive equivalga a 2, poiché ogni agente fornisce un contributo parziale.
- **epsilon:** Nel contesto dell'algoritmo di ascesa del gradiente applicato al controllo della copertura dinamica, ϵ rappresenta il *passo di salita*. Questo parametro è cruciale per l'aggiornamento delle posizioni degli agenti mobili, come i droni, ed è direttamente responsabile della velocità con cui gli agenti si muovono lungo la traiettoria determinata dal gradiente della funzione obiettivo.
 - **Definizione e ruolo:** ϵ è un parametro scalare positivo che determina l'entità del passo compiuto dagli agenti nella direzione del gradiente. In altre parole, controlla quanto grandi saranno gli aggiornamenti delle posizioni degli agenti a ogni iterazione dell'algoritmo.
 - **Impatto sulla convergenza:**
 - * **Valori piccoli:** Se ϵ è molto piccolo, gli aggiornamenti delle posizioni saranno molto gradualmente. Questo può garantire una convergenza più stabile e precisa verso il massimo della funzione obiettivo, ma al costo di un numero maggiore di iterazioni e, quindi, di un tempo di calcolo più lungo.
 - * **Valori grandi:** Se ϵ è troppo grande, gli agenti compiranno passi più ampi. Questo può accelerare la convergenza iniziale, ma rischia di far oscillare le posizioni degli agenti attorno al punto di massimo, senza mai raggiungerlo precisamente. In casi estremi, può portare alla divergenza dell'algoritmo.
 - **Selezione:** La scelta del valore di ϵ è fondamentale e spesso viene determinata attraverso esperimenti e tuning specifici per il problema in questione. In molti casi, un valore medio che bilancia la velocità di convergenza e la stabilità è preferibile. In alcuni approcci avanzati, ϵ può essere adattivo, variando dinamicamente durante l'esecuzione dell'algoritmo per ottimizzare il processo di apprendimento.

Nel progetto, il passo di salita ϵ è stato impostato a 150. Questa scelta è stata effettuata dopo aver effettuato varie prove con valori più piccoli e più grandi. Si è notato che un valore di 150 rappresenta il miglior compromesso per evitare sia la convergenza troppo lenta che l'oscillazione attorno al punto di massimo.

- **delta:** Il parametro δ nella versione 3 dell'algoritmo rappresenta la distanza di allontanamento tra gli agenti che definisce l'intensità della forza del potenziale repulsivo (vedi 2.3). In questa implementazione, δ è stato impostato a 30. La scelta di questo valore è basata su considerazioni sia teoriche che pratiche:
 - **Relazione con le distanze medie:** δ deve essere abbastanza grande da garantire che gli agenti non si avvicinino troppo l'uno all'altro, evitando collisioni. Deve essere proporzionato alla scala dello spazio in cui gli agenti si muovono. Nel nostro scenario, un'area ampia permette movimenti sufficienti, ma δ deve essere piccolo abbastanza da non compromettere l'efficacia del controllo della copertura.
 - **Esperimenti preliminari:** Sono stati condotti vari esperimenti con valori differenti di δ . Valori troppo piccoli risultavano in un comportamento inefficace del potenziale repulsivo, portando a collisioni frequenti. Valori troppo grandi, invece, tendevano a ridurre l'efficacia della copertura, poiché gli agenti evitavano l'un l'altro in modo eccessivo. Il valore di 30 è emerso come un compromesso ottimale, minimizzando le collisioni senza compromettere la copertura.
 - **Coefficiente di repulsione:** δ determina anche l'intensità della forza repulsiva. Una δ troppo alta potrebbe risultare in forze troppo deboli per essere efficaci, mentre una δ troppo bassa potrebbe causare forze eccessive, destabilizzando il sistema.

Nella prossima sottosezione verranno presentati grafici e risultati dell'implementazione del codice, come descritto in 3. La simulazione è stata eseguita utilizzando i valori dei parametri e le traiettorie specificate in precedenza. Questi risultati forniscono una valutazione completa dell'efficacia degli algoritmi di copertura dinamica implementati.

4.2 Analisi delle operazioni preliminari

Analizziamo i risultati delle operazioni preliminari, precedentemente descritte nella sezione 3.1:

1) Estrazione delle traiettorie dei targets:

Le traiettorie complete dei 10 targets scelti per la simulazione, di durata 150 secondi, sono riportate nel seguente grafico:

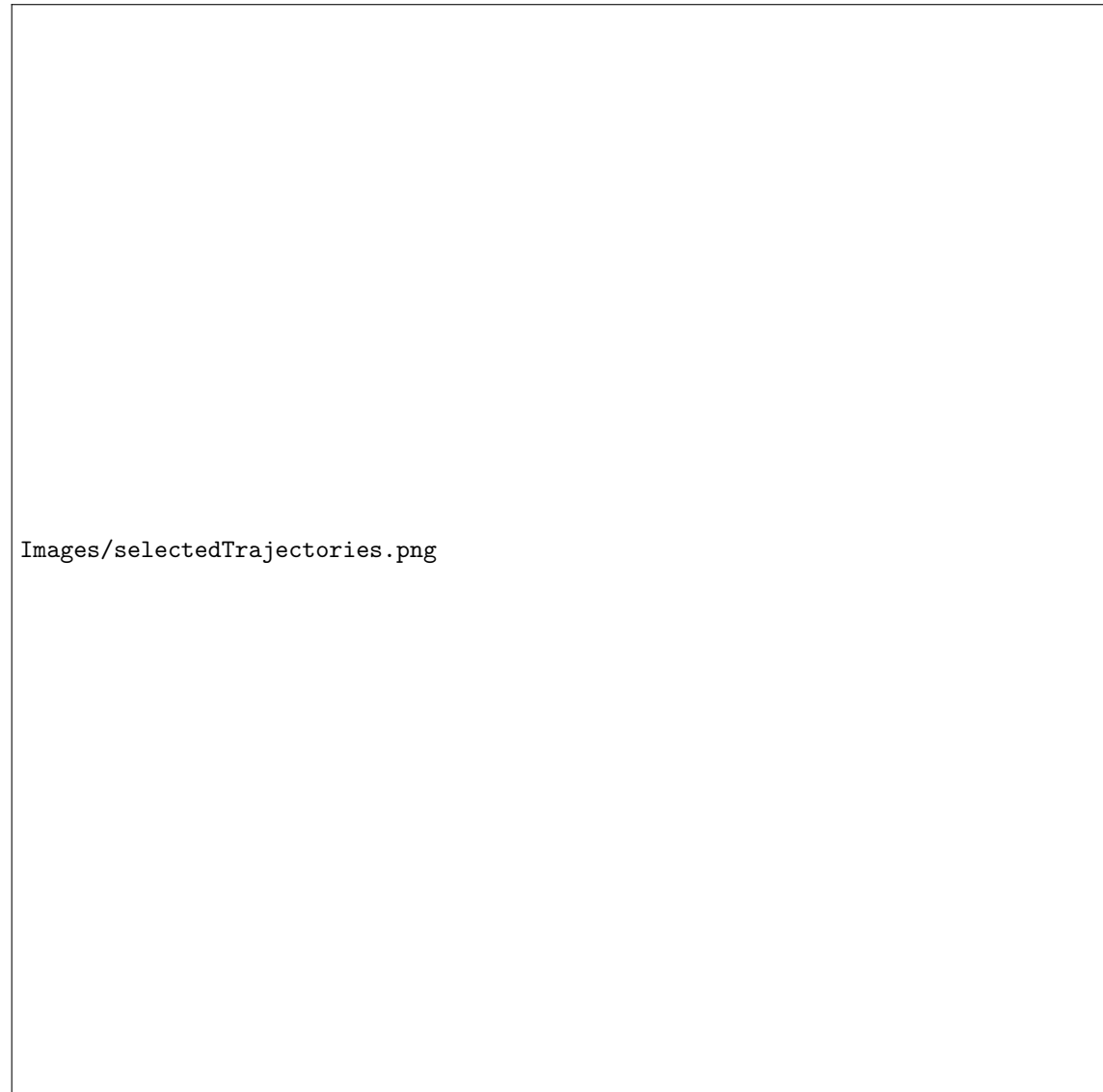


Figura 10: Traiettorie dei targets

2) Generazione posizioni iniziale degli agenti

Le posizioni iniziali degli agenti, essendo quattro, sono distribuite come segue:

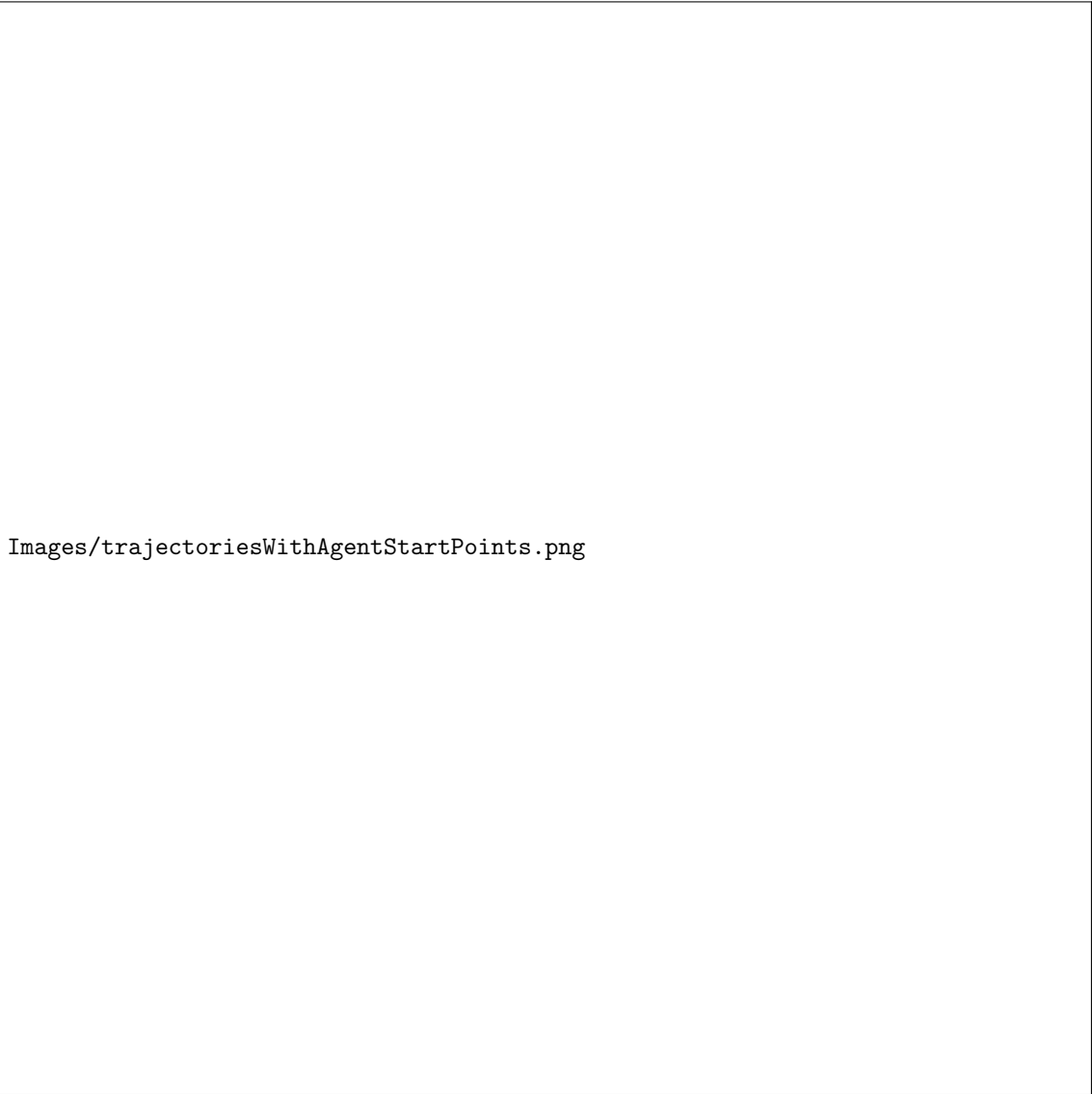
- Agente 1: $x = 50.0$, $y = 50.0$

- Agente 2: $x = 50.0$, $y = 150.0$
- Agente 3: $x = 150.0$, $y = 50.0$
- Agente 4: $x = 150.0$, $y = 150.0$



Figura 11: Posizioni iniziali degli agenti

Per fornire una visione d'insieme, è incluso un grafico che mostra le traiettorie degli agenti insieme alle posizioni iniziali dei target. Si può osservare che gli agenti partono da posizioni prossime ai punti di partenza dei target, facilitando l'inizio dell'operazione di copertura.



Images/trajectoriesWithAgentStartPoints.png

Figura 12: Traiettorie dei targets e posizioni iniziali degli agenti

3) Calcolo degli indici di copertura iniziali

Il calcolo degli indici di copertura iniziali è essenziale per confrontare i valori finali e valutare l'efficacia dell'algoritmo nell'incrementare la copertura.

Come si evince dalla tabella, al tempo iniziale $t = 0$, gli indici di copertura $E_j(0)$ per tutti i target j sono in media superiori a 1. Ciò indica che inizialmente ogni target è coperto da poco più di un agente.

Sotto viene riportato il valore iniziale dell'indice di copertura totale. Questo valore è fondamentale perché deve aumentare all'istante finale per dimostrare che la copertura dei target da parte degli agenti è migliorata.

j	$E_j(0)$
0	1.36690753
1	1.43775742
2	1.48737943
3	1.64403469
4	1.50434003
5	0.92326174
6	1.36251576
7	1.22878553
8	1.69180094
9	1.71275626

Tabella 1: Indici di copertura iniziali $E_j(t)$ al tempo $t = 0$

$E(0)$
2.53832315

Tabella 2: Valore iniziale dell'indice di copertura totale $E(0)$.

4.3 Analisi e grafici dei dati sperimentali

Dopo aver eseguito e testato le operazioni preliminari, si procede con l'implementazione delle tre versioni dell'algoritmo di copertura dinamica, oggetto di analisi del progetto. Questi algoritmi sono progettati per costruire le traiettorie degli agenti in modo tale che seguano correttamente i target e aumentino l'indice di copertura totale.

Di seguito i grafici completi che includono sia le traiettorie dei targets che quelle degli agenti, calcolate grazie all'esecuzione degli algoritmi:

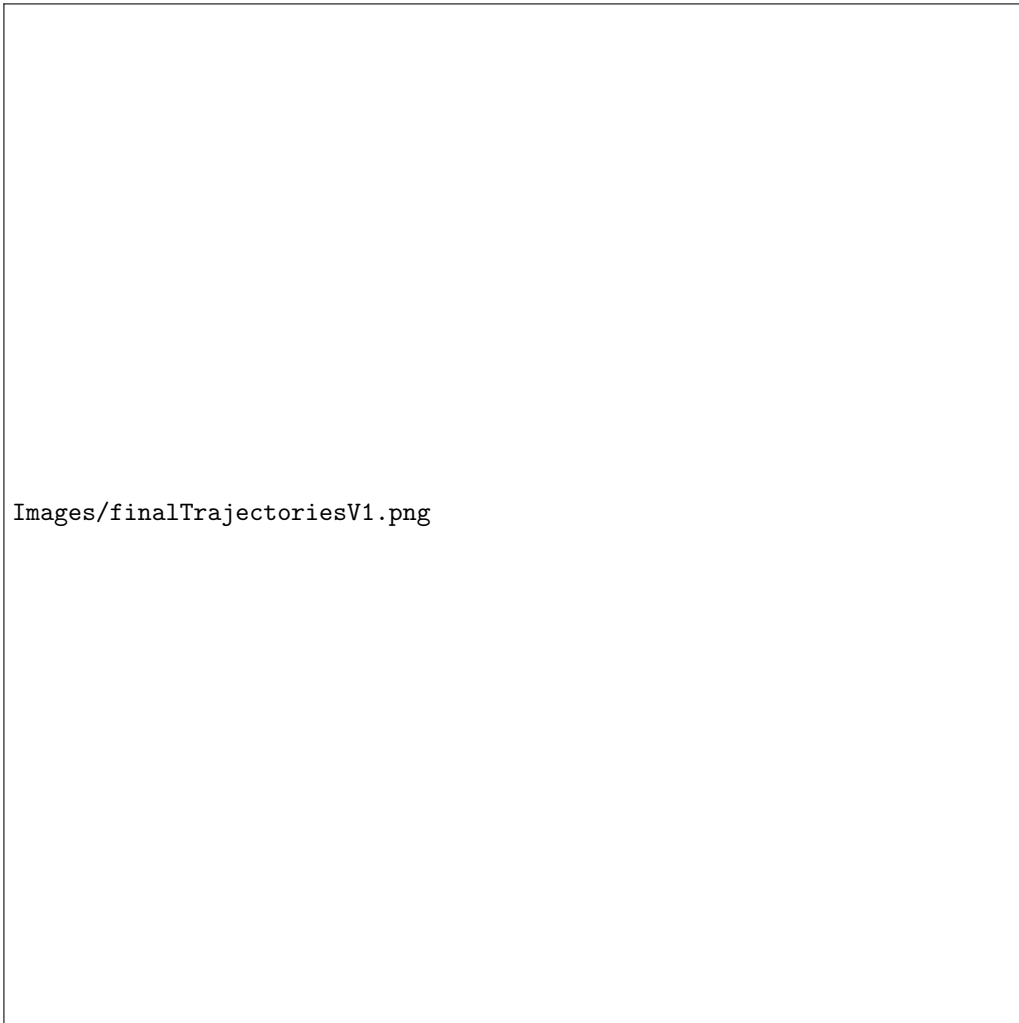


Figura 13: Traiettorie dei targets e degli agenti, V1

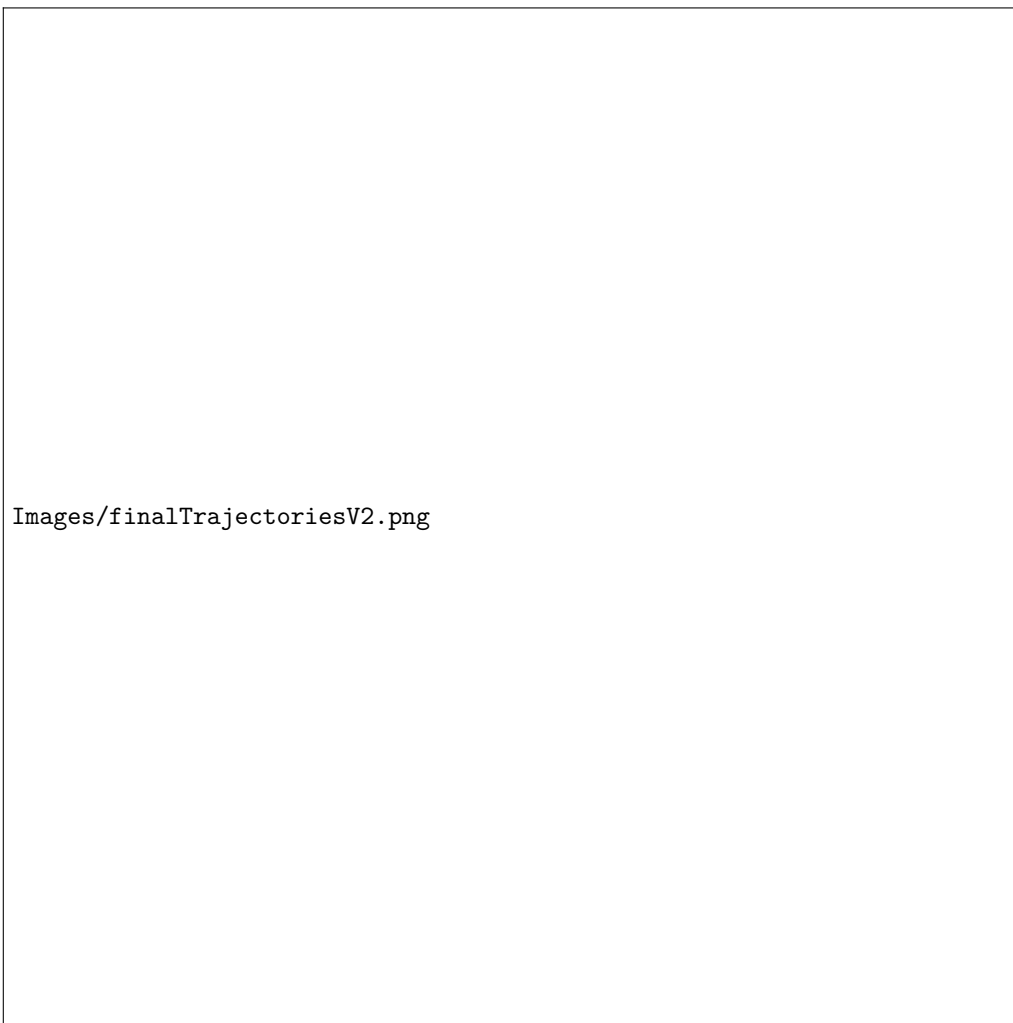


Figura 14: Traiettorie dei targets e degli agenti, V2

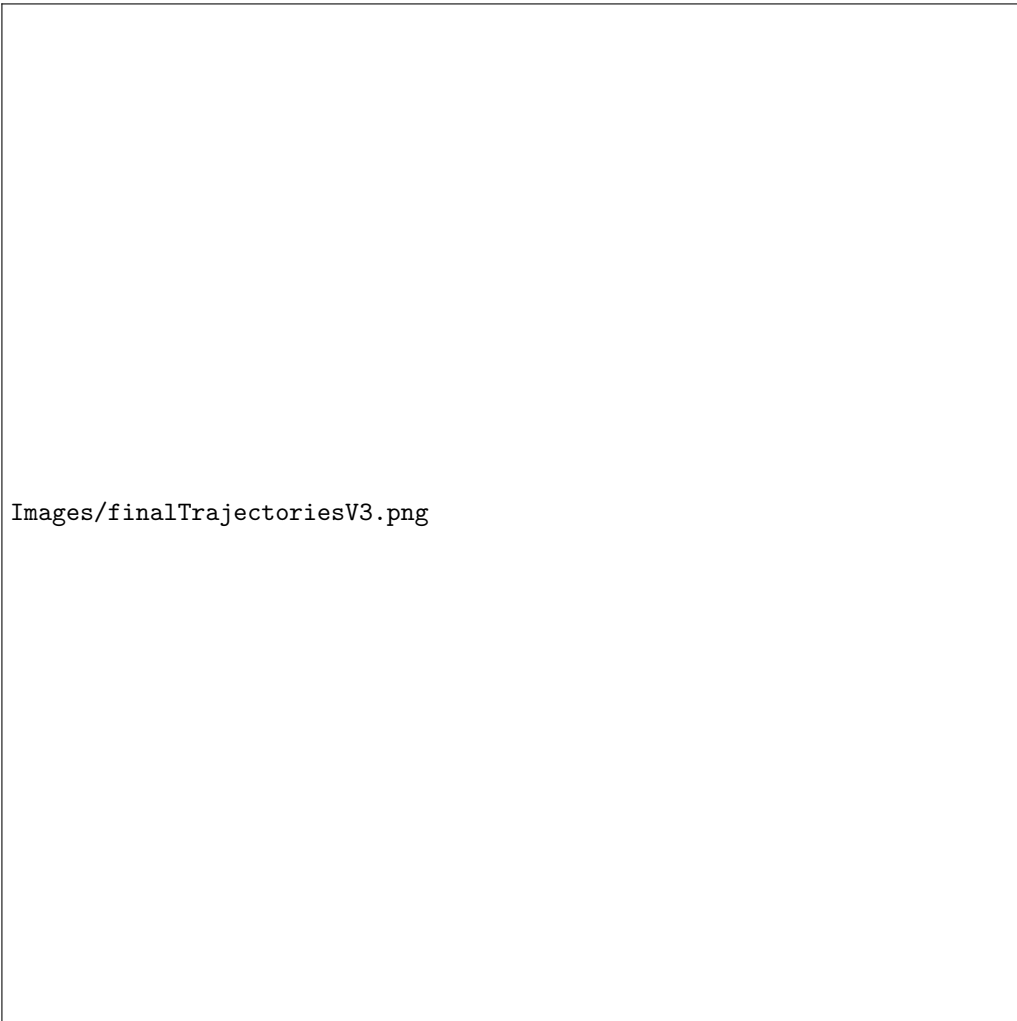


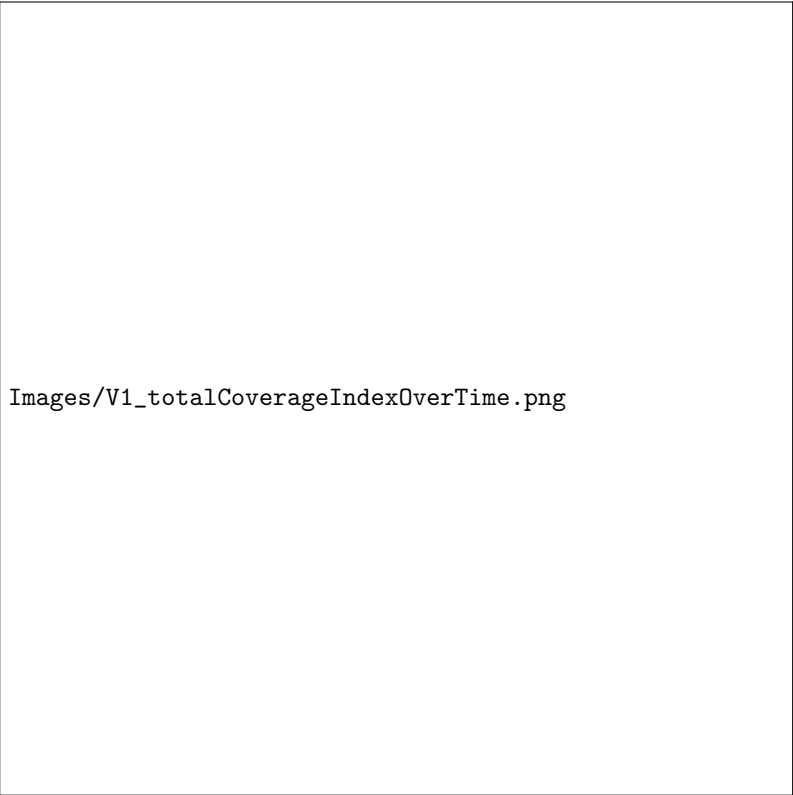
Figura 15: Traiettorie dei targets e degli agenti, V3

Dai grafici delle traiettorie complete dei target e degli agenti (figure 13, 14 e 15) per le tre versioni dell'algoritmo, si può osservare che:

1. Nella prima versione dell'algoritmo, descritta nella sezione 2.1, i target finiscono per *collidere* mentre cercano di massimizzare l'indice di copertura totale.
2. Nella seconda versione, il problema della collisione non viene risolto in modo soddisfacente. Sebbene l'aggiunta del moto Browniano introduca variazioni casuali nelle traiettorie, i target tendono comunque a rimanere troppo vicini, finendo spesso in collisione.
3. Nell'ultima versione, grazie all'introduzione del potenziale repulsivo, che entra in azione quando i target si avvicinano oltre una certa distanza δ , il problema della collisione viene efficacemente risolto. Questo risultato viene ottenuto senza compromettere la massimizzazione dell'indice di copertura totale, un aspetto che verrà analizzato più in dettaglio successivamente.

Per determinare se l'obiettivo del progetto è stato raggiunto, è necessario effettuare un confronto tra gli indici di copertura iniziali e finali e trarre delle conclusioni in merito.

Di seguito sono riportati i grafici che illustrano l'andamento degli indici di copertura calcolati per tutte e tre le versioni dell'algoritmo.



Images/V1_totalCoverageIndexOverTime.png

Figura 16: Indice di copertura totale $E(t)$, V1



Figura 17: Indice di copertura totale $E(t)$, V2



Figura 18: Indice di copertura totale $E(t)$, V3

Analizziamo anchei valori iniziali e finali di questi tre diversi indici di copertura:

	Valore Iniziale ($E(0)$)	Valore Finale ($E(\text{duration}-1)$)
Algoritmo V1	2.5383	6.3206
Algoritmo V2	2.5383	6.3725
Algoritmo V3	2.5383	6.2578

Tabella 3: Confronto tra i valori iniziali e finali dell'indice di copertura totale

Grazie all'analisi dei grafici e alla tabella di confronto, è possibile confermare che il progetto è stato portato a termine con successo e che gli algoritmi di coverage dinamico implementati soddisfano pienamente le aspettative iniziali:

1. L'andamento del grafico dell'indice di copertura totale è crescente in tutte e tre le versioni dell'algoritmo. Il valore finale è significativamente maggiore rispetto al valore iniziale, indicando un notevole miglioramento nella copertura totale dell'area.
2. I valori degli indici di copertura finali per le versioni 2 e 3 dell'algoritmo sono molto simili a quelli della versione originale. Questo risultato dimostra che la versione 3 dell'algoritmo è riuscita a evitare la collisione finale degli agenti, pur mantenendo un'elevata copertura dell'area e massimizzando l'indice di copertura. Questo aspetto conferma l'efficacia del metodo del potenziale repulsivo, descritto nella sezione 2.3.

Per concludere, è importante andare anche ad analizzare il comportamento dei singoli indici di copertura dei target:

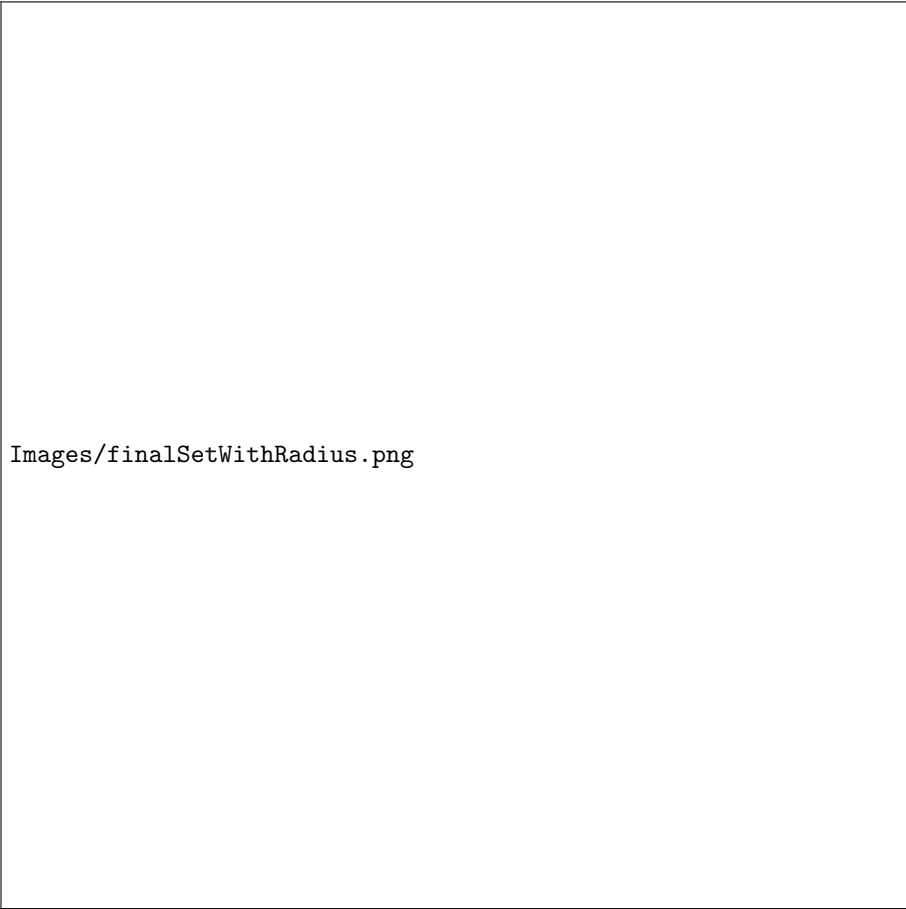
Target j	Iniziale $E_j(0)$	Finale V1 $E_j(150)$	Finale V2 $E_j(150)$	Finale V3 $E_j(150)$
$j = 0$	1.3669	0.0000	0.0038	0.0264
$j = 1$	1.4378	2.9858	3.3193	2.8690
$j = 2$	1.4874	3.5171	3.1065	3.4770
$j = 3$	1.6440	3.4701	3.4722	3.3872
$j = 4$	1.5043	3.6248	3.7138	3.5210
$j = 5$	0.9233	2.3512	1.8834	2.3811
$j = 6$	1.3625	3.8787	3.8085	3.7653
$j = 7$	1.2288	0.5045	1.0068	0.4871
$j = 8$	1.6918	0.0000	0.0000	0.0000
$j = 9$	1.7128	2.8539	2.6135	2.7741

Tabella 4: Confronto degli indici di copertura iniziali e finali per le tre versioni dell'algoritmo

Come è possibile osservare dalla differenza tra i valori iniziali e quelli finali, la tesi analizzata nella sezione 1.2 è confermata. Inizialmente, ogni target è coperto in media da poco più di un agente, poiché gli indici iniziali sono leggermente superiori a 1 (si veda il ragionamento sui parametri analizzato in 4.1). Tuttavia, in tutte e tre le versioni, all'istante finale, gli indici di copertura del target 1 ($j=0$) e del target 9 ($j=8$) sono praticamente azzerati. Questo comportamento non è indesiderato, poiché l'algoritmo di copertura punta a massimizzare l'indice di copertura globale. Se per raggiungere questo obiettivo è necessario sacrificare alcuni target riducendo drasticamente la loro copertura, l'algoritmo lo fa. Questo comportamento può essere verificato osservando i grafici 13, 14 e 15, in cui si nota che i target azzurro e verdolino, nelle loro posizioni finali, sono molto distanti dalle posizioni finali degli agenti, addirittura fuori dal loro raggio di copertura. Gli agenti, seguendo la logica dell'algoritmo, tenderanno a concentrarsi nelle posizioni con una maggiore densità di target, al fine di massimizzare la funzione obiettivo, ovvero l'indice di copertura totale (**vedi figura 19**). *Inoltre, solo 2 dei 10 target sono coperti in modo non soddisfacente, un risultato che risulta più che accettabile.*

È importante notare anche che i valori degli **indici di copertura finali** della maggior parte dei target sono nettamente superiori al valore di soglia E^* pari a 2. Questo indica che i suddetti target avranno una copertura finale eccellente, confermando quanto affermato nella sezione 1.2.

Inoltre, i valori degli indici di copertura finali nelle tre diverse versioni dell'algoritmo di coverage sono piuttosto simili. Questo comportamento indica che, nonostante l'aggiunta di un moto browniano (2.2) o di un potenziale repulsivo (2.3), la qualità dell'algoritmo non è stata compromessa, mantenendo il trend principale dell'ascesa del gradiente. Nella versione 2, i cambiamenti dei valori rispetto alla versione classica sono molto casuali, dovuti alle componenti stocastiche del moto. Nella versione 3, invece, i valori sono leggermente inferiori, a favore però della non collisione finale degli agenti.



Images/finalSetWithRadius.png

Figura 19: Posizioni finali di targets e agenti con raggi di copertura

Come ultima e importante conferma, il grafico che mostra le posizioni finali dei target e degli agenti con il loro raggio di copertura (traiettorie ottenute con la versione 3 dell'algoritmo) evidenzia che i target 1, 5 e 9 sono quelli coperti peggio. Confrontando questo grafico con la tabella 4, *si può affermare che i dati e i risultati ottenuti sono coerenti con le aspettative teoriche:*

1. Il target 9 ($j=0$) è completamente fuori dai raggi di copertura degli agenti; infatti, l'indice finale è sempre pari a 0 in tutte e tre le versioni.
2. Il target 1 ($j=1$) è completamente fuori dai raggi di copertura nella prima versione dell'algoritmo, dove l'indice è pari a 0. Nelle altre due versioni è coperto in minima parte dal terzo agente, con un indice di copertura leggermente superiore a 0.
3. Il target 6 ($j=5$) è coperto in modo meno efficiente rispetto agli altri. Tuttavia, il suo indice di copertura è mediamente superiore alla soglia $E^* = 2$.

4.4 Richiesta di coverage meno restrittiva

In questa sottosezione sarà analizzato il caso in cui il parametro $E^* = lowerBoundIndex$ è impostato a 1 anziché a 2. Questo implica una *richiesta di copertura inferiore*: non è più necessario che almeno due agenti, in media, coprano un target, ma è sufficiente che sia coperto da un solo agente (vedi anche la sezione 1.2).

Verranno esaminate tutte le conseguenze di questa scelta, replicando le considerazioni effettuate per il caso con $E^* = 2$ e confrontandole con quest'ultimo scenario.

Considerando le operazioni preliminari già eseguite, le traiettorie dei targets e degli agenti, ottenute dopo l'esecuzione del codice, sono illustrate nei seguenti grafici per tutte e tre le versioni dell'algoritmo:

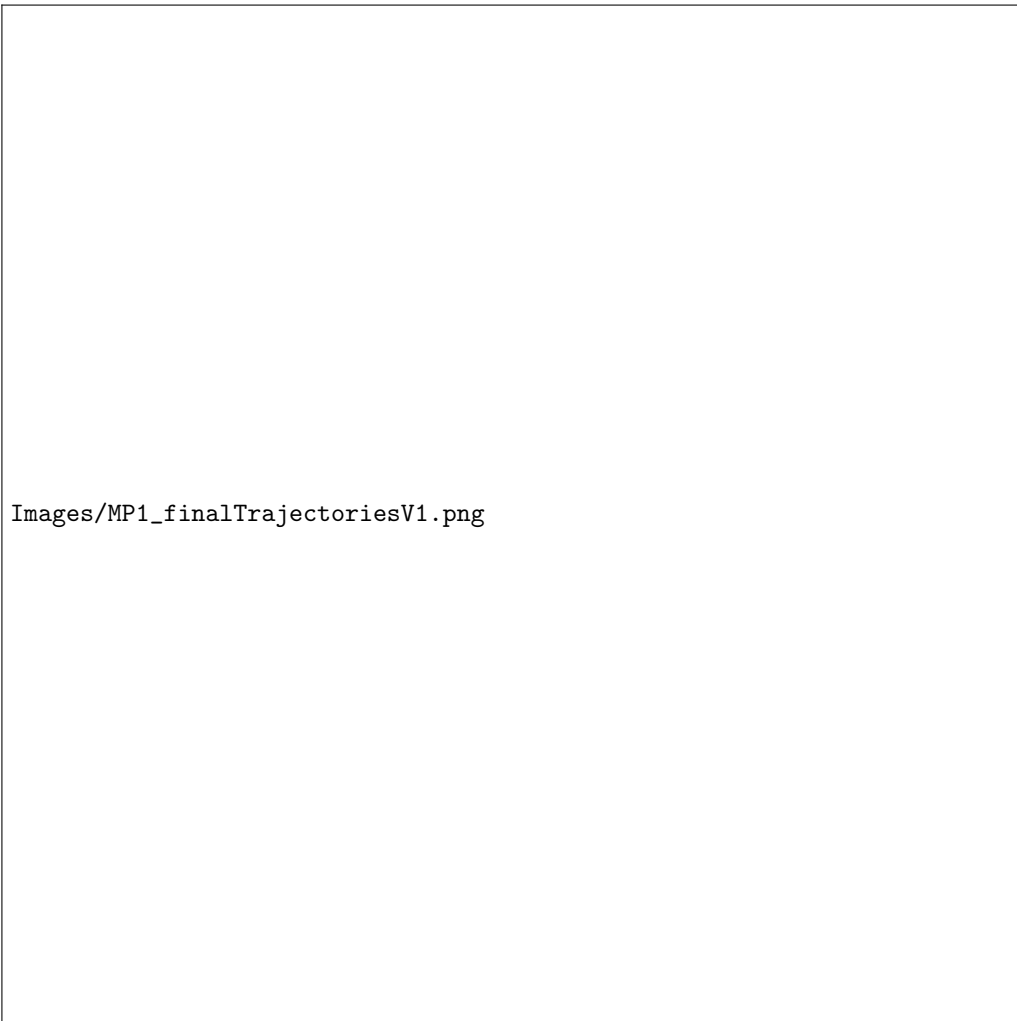
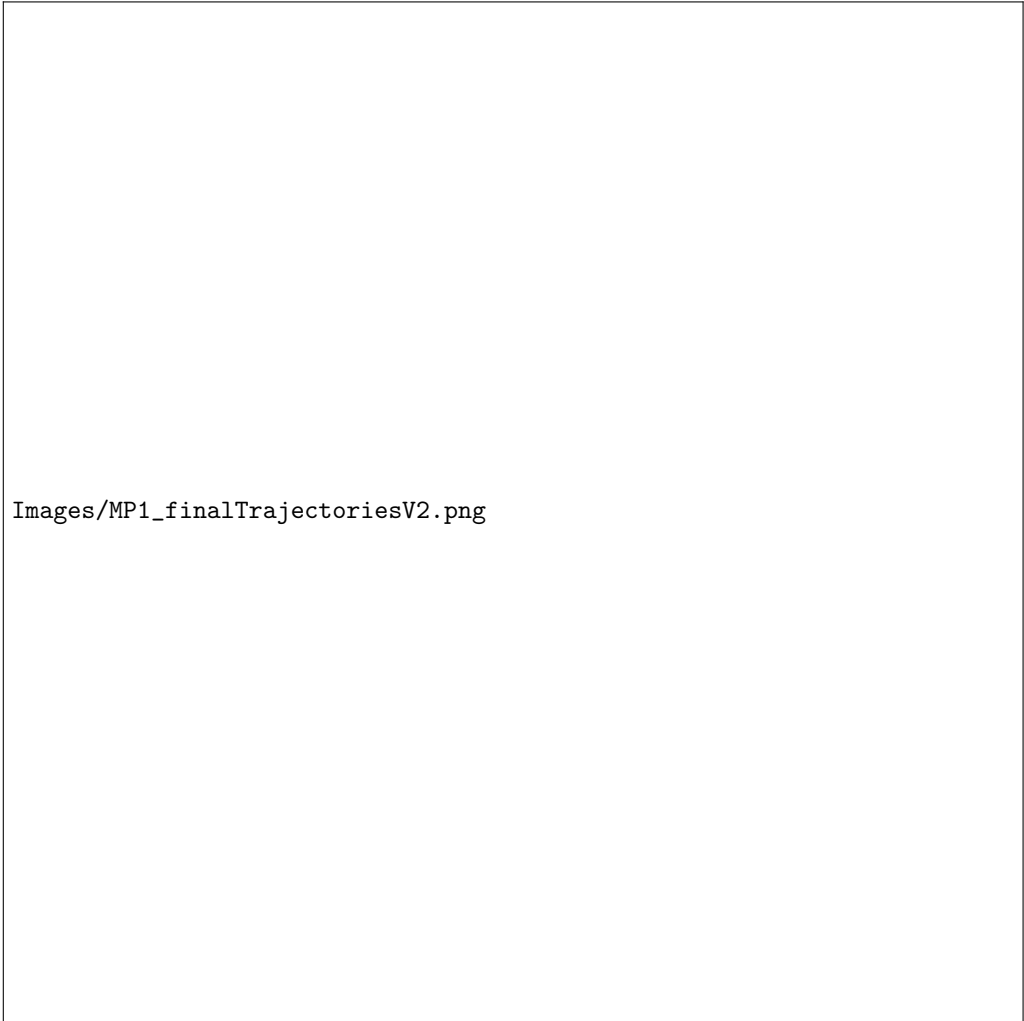


Figura 20: Traiettorie dei targets e degli agenti, V1, $E^*=1$



Images/MP1_finalTrajectoriesV2.png

Figura 21: Traiettorie dei targets e degli agenti, V2, $E^*=1$



Figura 22: Traiettorie dei targets e degli agenti, V3, $E^*=1$

Come si evince dalle figure 20, 21 e 22, a differenza dei grafici generati con E^* impostato a 2 (figure 13, 14 e 15), emerge una notevole differenza: le posizioni finali degli agenti risultano significativamente diverse e molto più distanti tra loro.

Questo comportamento è perfettamente logico e conferma l'importanza del parametro E^* . Con E^* impostato alla metà del suo valore precedente, la richiesta di copertura è ridotta. Di conseguenza, è naturale che gli agenti non si avvicinino fino alla collisione. Dato che è necessario coprire i target con minore intensità, gli agenti possono permettersi di rimanere più distanti tra loro. Questo permette una copertura leggermente inferiore per i target precedentemente ben coperti e una copertura migliorata per i target che prima erano completamente fuori dal raggio di visione degli agenti, garantendo comunque una copertura complessivamente soddisfacente.

A conferma di questa tesi, si può osservare il grafico delle posizioni finali degli agenti e dei target, che mostra anche i raggi di copertura dei droni, e confrontarlo con il grafico precedente (19):

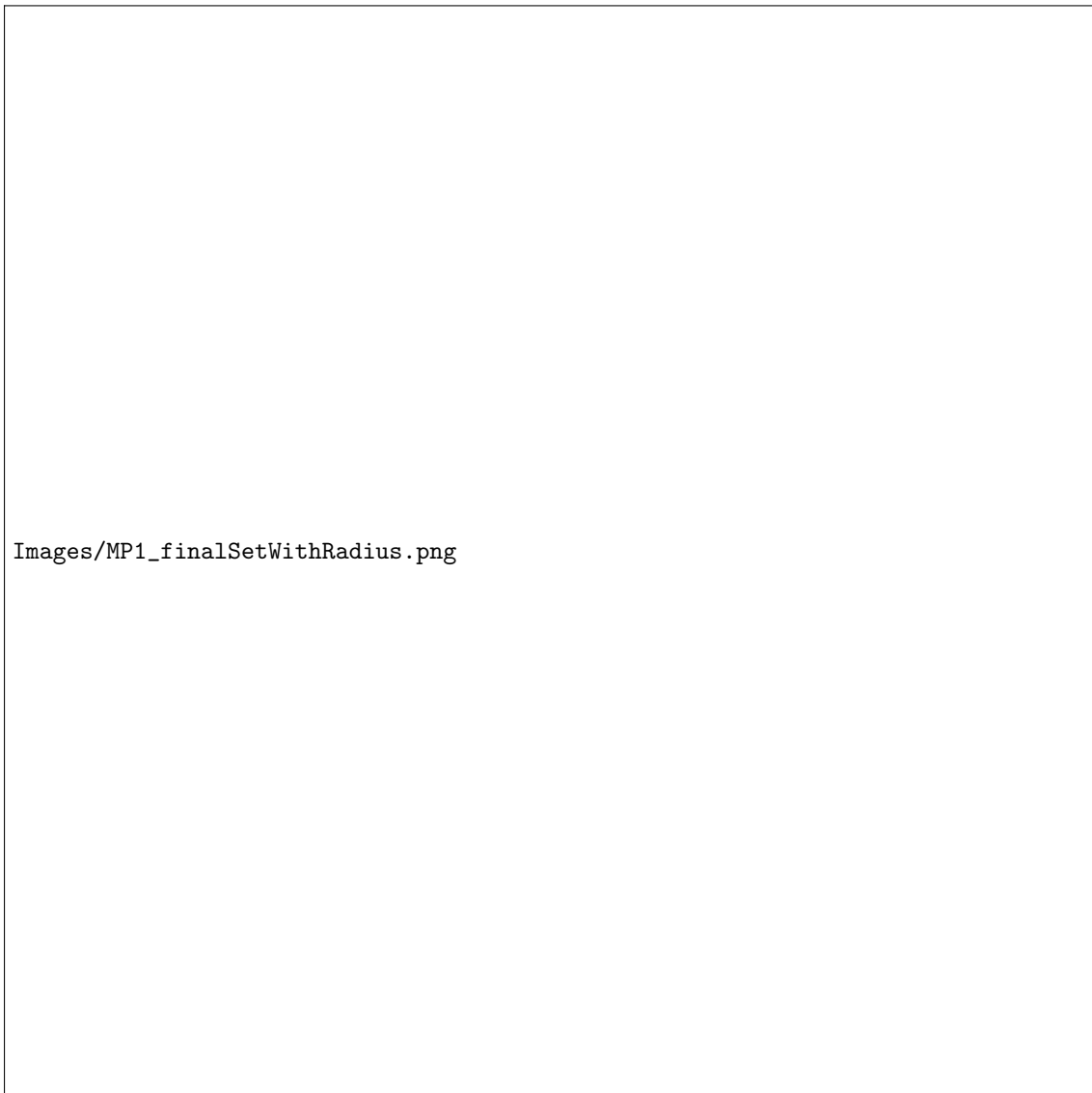


Figura 23: Posizioni finali di targets e agenti con raggi di copertura, $E^*=1$

È fondamentale esaminare anche le differenze nei valori dei singoli indici di copertura e dell'indice di copertura totale:

Target j	Iniziale $E_j(0)$	Finale V1 $E_j(150)$	Finale V2 $E_j(150)$	Finale V3 $E_j(150)$
$j = 0$	1.3669	0.9107	0.9730	0.9051
$j = 1$	1.4378	2.1927	2.1866	2.1717
$j = 2$	1.4874	2.4315	2.3223	2.4179
$j = 3$	1.6440	2.3590	2.3009	2.3455
$j = 4$	1.5043	2.6228	2.5759	2.5999
$j = 5$	0.9233	1.3669	1.2959	1.3758
$j = 6$	1.3625	3.3449	3.2869	3.3143
$j = 7$	1.2288	0.6475	0.7223	0.6294
$j = 8$	1.6918	0.0217	0.0209	0.0613
$j = 9$	1.7128	2.7822	2.7140	2.7725

Tabella 5: Confronto degli indici di copertura iniziali e finali per le tre versioni dell'algoritmo, $E^*=1$

Come si può osservare, i valori degli indici di copertura sono inferiori rispetto a quelli riportati nella tabella 4. Questo risultato è coerente, poiché la copertura richiesta è minore. Come descritto in precedenza, la copertura fornita è eccellente (superiore a $E^* = 1$) per quasi tutti i target, ad eccezione dei tre target che erano già parzialmente coperti. Tuttavia, anche questi target marginali ora godono di una copertura migliore, con indici vicini alla soglia di 1 (vantaggio principale di andare ad essere meno restrittivi nella richiesta del coverage). Infatti, si può osservare che già all'istante iniziale si ottiene praticamente una copertura completa, con indici di copertura quasi tutti superiori alla soglia.

Ci sono delle differenze sostanziali anche per quanto riguarda l'andamento dell'indice di copertura totale:

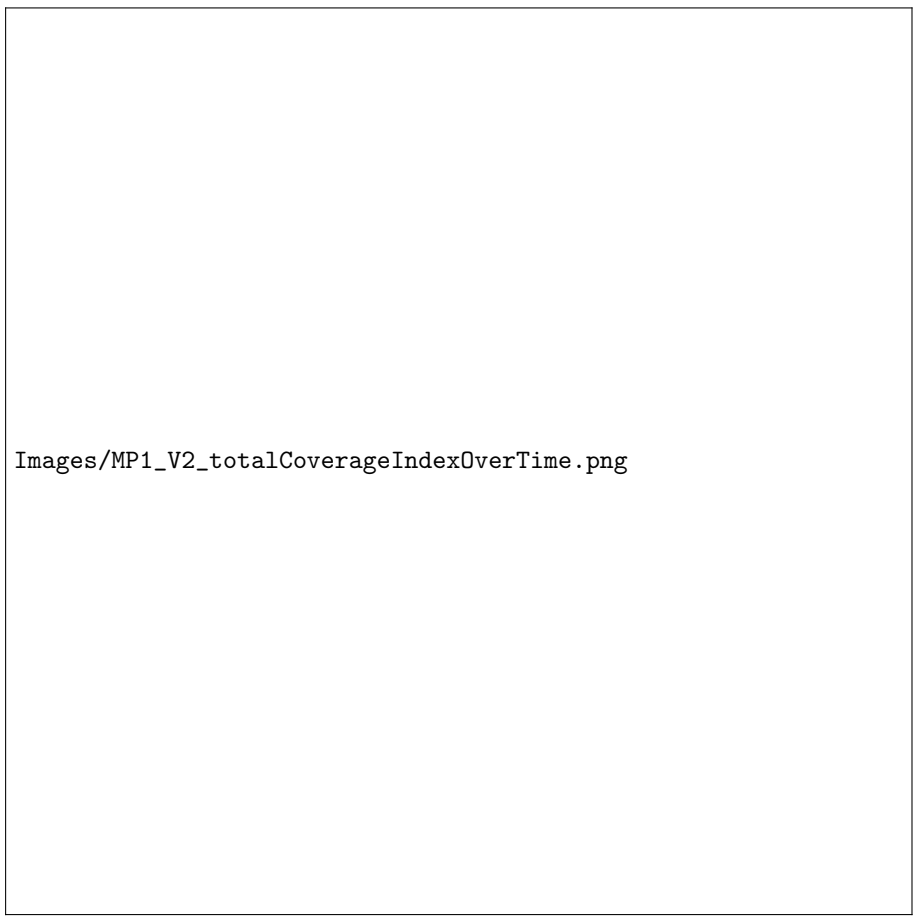
	Valore Iniziale ($E(0)$)	Valore Finale ($E(\text{duration}-1)$)
Algoritmo V1	6.9778	7.3113
Algoritmo V2	6.9778	7.3153
Algoritmo V3	6.9778	7.3042

Tabella 6: Confronto tra i valori iniziali e finali dell'indice di copertura totale, $E^*=1$

Giustamente, il valore iniziale dell'indice di copertura totale è maggiore rispetto a quello riportato nella tabella 3. Questo avviene perché, con una richiesta di copertura inferiore per i singoli target, la copertura totale iniziale risulta migliorata. Anche se l'indice di copertura totale aumenterà all'istante finale, questo incremento non sarà così drastico come nel caso precedente con la soglia pari a 2. Questa osservazione è confermata dai grafici seguenti:



Figura 24: Indice di copertura totale $E(t)$, V1, $E^*=1$



Images/MP1_V2_totalCoverageIndexOverTime.png

Figura 25: Indice di copertura totale $E(t)$, V2, $E^*=1$

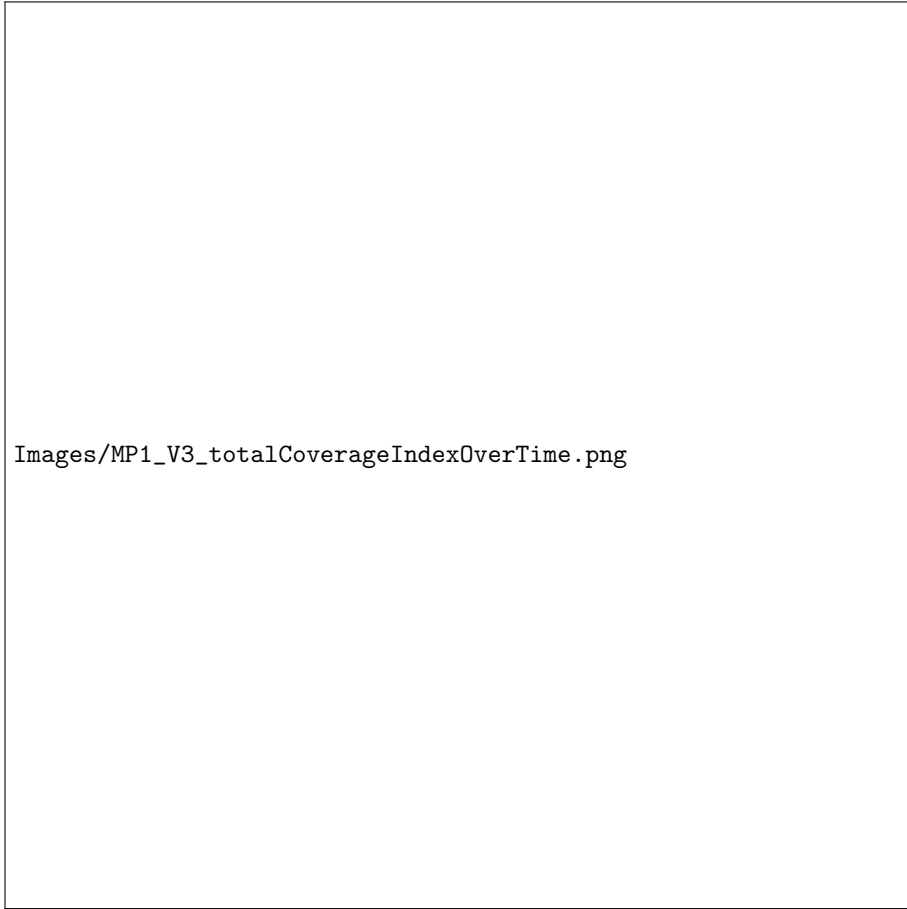


Figura 26: Indice di copertura totale $E(t)$, V3, $E^*=1$

Come evidenziato dai grafici precedenti, l'andamento dell'indice di copertura totale differisce rispetto al caso precedente. In questo scenario, si osserva una fase di crescita seguita da una fase di stabilizzazione. Nonostante ciò, il valore finale dell'indice di copertura totale risulta comunque superiore rispetto al caso con $E^* = 2$. Questo miglioramento della copertura totale è dovuto alla soglia inferiore. Pertanto, *è fondamentale comprendere il contesto specifico per trovare un compromesso ottimale tra il numero di target, il numero di agenti e la copertura richiesta.*

Conclusioni

In questa tesi di laurea, è stato affrontato il problema del controllo della copertura dinamica per il monitoraggio cooperativo in tempo reale utilizzando una flotta di agenti mobili. **Gli algoritmi sviluppati sono stati implementati e testati in un ambiente simulato, dimostrando la loro efficacia nel garantire un certo livello di copertura totale** fornito dagli agenti sui target dinamici.

L'obiettivo primario di questi algoritmi è mantenere il più alto livello possibile di copertura totale per tutta la durata del monitoraggio, adattandosi dinamicamente ai cambiamenti nell'ambiente e nelle posizioni dei target. Gli algoritmi sono progettati per ottimizzare continuamente la copertura, cercando di aumentarla nel tempo e garantendo che il sistema multi-agente operi in modo efficiente e privo di collisioni.

I risultati ottenuti confermano che tutti gli algoritmi sviluppati hanno migliorato significativamente l'indice di copertura totale rispetto allo stato iniziale. In particolare:

1. La prima versione dell'algoritmo ha mostrato un notevole miglioramento dell'indice di copertura totale, ma ha presentato problemi di collisione tra gli agenti nelle posizioni finali.
2. Nella seconda versione, l'introduzione del moto browniano ha mitigato parzialmente il problema delle collisioni, sebbene non in modo completamente soddisfacente.
3. Nell'ultima versione, l'inclusione del potenziale repulsivo ha risolto efficacemente il problema delle collisioni senza compromettere l'aumento dell'indice di copertura totale, garantendo una distribuzione spaziale ottimale degli agenti.

L'analisi dettagliata degli indici di copertura in funzione del tempo ha evidenziato che l'algoritmo riesce ad aumentare e garantire nel tempo un alto livello di copertura per i target che si muovono all'interno dell'area di sorveglianza.

Inoltre, è stato analizzato l'effetto della diminuzione della richiesta di copertura sull'indice di copertura totale. In particolare, con una richiesta di copertura inferiore, gli agenti possono distribuire le loro risorse in modo più efficiente, portando a un aumento complessivo dell'indice di copertura totale. Questa osservazione suggerisce che la flessibilità nella richiesta di copertura può condurre a una gestione più efficiente delle risorse degli agenti, migliorando ulteriormente la qualità del monitoraggio.

In conclusione, i risultati di questa tesi dimostrano che gli algoritmi sviluppati forniscono una soluzione robusta ed efficace per il controllo della copertura dinamica, con potenziali applicazioni in numerosi campi, dalla sorveglianza alla gestione delle emergenze.

Ecco una lista di alcuni **possibili sviluppi** di questa tesi di laurea:

1. Un primo sviluppo futuro consiste nell'andare oltre l'approccio greedy basato su informazioni istantanee, integrando la predizione delle traiettorie dei target tramite tecniche avanzate come Reinforcement Learning (RL), Deep Reinforcement Learning (DRL), Multi-Agent Deep Reinforcement Learning (MADRL) e approcci model-based. Il RL permette agli agenti di apprendere politiche ottimali massimizzando le ricompense cumulative (24), mentre il DRL utilizza reti neurali profonde per apprendere in spazi di stato e azione continui (25). Il MADRL estende questi concetti a scenari multi-agente, ottimizzando la cooperazione e la competizione tra agenti (26). Gli approcci model-based costruiscono modelli delle transizioni di stato per migliorare la predizione delle dinamiche temporali e l'efficienza dell'apprendimento (27).
2. Un secondo sviluppo futuro potrebbe riguardare la modifica dell'altitudine degli agenti, poiché nella progetto attuale essa è fissata. Un'alterazione dell'altitudine permette agli agenti di coprire un'area maggiore, se volano più in alto, ma a scapito di una risoluzione inferiore delle immagini o dei dati raccolti. La sfida risiede nel trovare un compromesso ottimale tra estensione dell'area coperta e qualità della risoluzione. Tecniche di ottimizzazione multi-obiettivo potrebbero essere impiegate per bilanciare questi due aspetti, garantendo che la copertura sia sufficientemente ampia senza compromettere eccessivamente la risoluzione dei dati (28), (29). Questo approccio potrebbe essere ulteriormente migliorato integrando modelli predittivi che aiutino a decidere l'altitudine ottimale in tempo reale, in base alla dinamica dei target e alle esigenze di risoluzione (30).
3. Un terzo possibile sviluppo riguarda l'esplorazione dell'area di interesse. Nel progetto di tesi, si assume di conoscere a priori il punto di partenza dei target, le loro traiettorie e il numero preciso di target presenti nella zona di analisi. Tuttavia, in scenari reali, queste informazioni potrebbero non essere disponibili. Pertanto, un'estensione del lavoro potrebbe consistere nell'integrare una fase di esplorazione, in cui gli agenti effettuano una ricognizione dell'area per raccogliere informazioni sulla posizione e sul numero dei target. Questa esplorazione potrebbe utilizzare tecniche di SLAM (Simultaneous Localization and Mapping) per mappare l'ambiente e individuare i target (31), (32).
L'esplorazione non solo aiuterebbe a mappare l'area e a identificare i target, ma potrebbe anche rilevare la presenza di intrusi o oggetti non previsti. Metodi avanzati di esplorazione autonoma, basati su approcci di ottimizzazione e tecniche probabilistiche, possono migliorare l'efficienza e la precisione di questa fase (33).

Bibliografija

- [1] J. Cortes, S. Martinez, T. Karatas and F. Bullo, "Coverage control for mobile sensing networks," in IEEE Transactions on Robotics and Automation, vol. 20, no. 2, pp. 243-255, April 2004, doi: 10.1109/TRA.2004.824698. keywords: Sensor phenomena and characterization;Remotely operated vehicles;Mobile robots;Temperature sensors;Biosensors;Animals;Optimization methods;Partitioning algorithms;Prototypes;Infrared sensors, <https://ieeexplore.ieee.org/document/1284411>
- [2] S. Meng and Z. Kan, "Deep Reinforcement Learning-Based Effective Coverage Control With Connectivity Constraints," in IEEE Control Systems Letters, vol. 6, pp. 283-288, 2022, doi: 10.1109/LCSYS.2021.3070850. keywords: Sensors;Monitoring;Dynamics;Task analysis;Reinforcement learning;Vehicle dynamics;Training;Coverage control;deep reinforcement learning;multi-agent systems, <https://ieeexplore.ieee.org/document/9395182>
- [3] Hübel, Nico. (2008). Coverage Control with Information Decay in Dynamic Environments. IFAC Proceedings Volumes. 41. 4180-4185. 10.3182/20080706-5-KR-1001.00703. https://www.researchgate.net/publication/254995455_Coverage_Control_with_Information_Decay_in_Dynamic_Environments
- [4] Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press. <https://www.deeplearningbook.org/>
- [5] Stipanović, D.M., Hokayem, P.F., Spong, M.W., & Šiljak, D.D. (2007). Cooperative avoidance control for multi-agent systems. *Journal of Dynamic Systems, Measurement, and Control*, 129, 699-707. doi:10.1115/1.2764502. <https://asmedigitalcollection.asme.org/dynamicsystems/article-abstract/129/5/699/446752/Cooperative-Avoidance-Control-for-Multiagent>
- [6] E. Nelson, *Dynamical Theories of Brownian Motion*, Princeton University Press, 1967. <https://web.math.princeton.edu/~nelson/books/bmotion.pdf>
- [7] M. Abdechiri, M. R. Meybodi, H. Bahrami, *Gases Brownian motion optimization: an algorithm for optimization (GBMO)*, Applied Soft Computing, 2013. <https://www.sciencedirect.com/science/article/pii/S1568494612001834>
- [8] K. Sowjanya, S. K. Injeti, *Investigation of butterfly optimization and gases Brownian motion optimization algorithms for optimal multilevel image thresholding*, Expert Systems with Applications, 2021. <https://www.sciencedirect.com/science/article/pii/S095741742100717X>
- [9] Q. Xiaohong, Q. Xiaohui, *An evolutionary particle swarm optimizer based on fractal Brownian motion*, Journal of Computational Intelligence, 2012. <https://www.ingentaconnect.com/contentone/asp/jcies/2012/00000001/00000001/art00023>

- [10] H. Hao, et al., *Potential Field Method for Multi-Agent Formation Control with Collision Avoidance*, MDPI, 2020.
<https://www.mdpi.com/2076-3417/10/20/7259>
- [11] J. Jiang, et al., *Enhanced Dynamic Coverage Using Repulsive Potentials in Multi-Agent Systems*, MDPI, 2021.
<https://www.mdpi.com/2079-9292/10/8/950>
- [12] I. Nagy, *Behaviour Study of a Multi-Agent Mobile Robot System during Potential Field Building*, Acta Polytechnica Hungarica, 2009.
https://www.researchgate.net/publication/45087275_Behaviour_Study_of_a_Multi-Agent_Mobile_Robot_System_during_Potential_Field_Building
- [13] R. Arkin, K. Ali, *Integration of Reactive and Telerobotic Control in Multi-Agent Robotic Systems*, Proc. Simulation of Adaptive Behavior, 1994.
<https://www-robotics.jpl.nasa.gov/media/documents/sab94.pdf>
- [14] M.H. Mabrouk, C.R. McInnes, *Social Potential Model to Simulate Emergent Behaviour for Swarm Robots*, 13th International Conference on Climbing and Walking Robots, 2008.
<https://strathprints.strath.ac.uk/7528/6/strathprints007528.pdf>
- [15] <https://docs.python.it/html/lib/module-pickle.html>
- [16] Joe D. Hoffman, *Numerical Methods for Engineers and Scientists*, 2nd Edition, CRC Press, 2001.
<https://www.crcpress.com/Numerical-Methods-for-Engineers-and-Scientists-Second-Edition/Hoffman/p/book/9780824704438>
- [17] James F. Epperson, *An Introduction to Numerical Methods and Analysis*, 2nd Edition, Wiley, 2013.
<https://www.wiley.com/en-us/An+Introduction+to+Numerical+Methods+and+Analysis%2C+2nd+Edition-p-9781118367599>
- [18] NumPy Developers, *NumPy: The fundamental package for scientific computing with Python*, <https://numpy.org/doc/stable/>.
- [19] Python Software Foundation, *pickle - Python object serialization*, <https://docs.python.org/3/library/pickle.html>.
- [20] John D. Hunter, *Matplotlib: A 2D Graphics Environment*, 2007. <https://matplotlib.org/stable/contents.html>
- [21] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, Adam Lerer. *Automatic differentiation in PyTorch*, 2017. <https://pytorch.org>.
- [22] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. <https://mitpress.mit.edu/books/introduction-algorithms-third-edition>

- [23] J. Kleinberg and É. Tardos, *Algorithm Design*. Pearson, 2006. <https://www.pearson.com/store/p/algorithm-design/P100000052280>
- [24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT press, 2018. <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [25] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. <https://www.nature.com/articles/nature14236>
- [26] R. Lowe et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in Neural Information Processing Systems*, vol. 30, 2017. <https://proceedings.neurips.cc/paper/2017/file/68a9750337a418a86fe06c1991a1d64c-Paper.pdf>
- [27] K. Chua et al., "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," *Advances in Neural Information Processing Systems*, vol. 31, 2018. <https://proceedings.neurips.cc/paper/2018/file/3de568f8597b94bda53149c7d7f8b16c-Paper.pdf>
- [28] Y. Zhao, L. Meng, and S. Wang, "Multi-objective optimization for coverage path planning in aerial remote sensing," *IEEE Access*, vol. 7, pp. 45606–45619, 2019. <https://ieeexplore.ieee.org/document/8695680>
- [29] R. Beard and T. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2020. <https://press.princeton.edu/books/hardcover/9780691149219/small-unmanned-aircraft>
- [30] L. Merino, F. Caballero, J. R. Martínez-de-Dios, I. Maza, and A. Ollero, "A cooperative perception system for multiple UAVs: Application to automatic detection of forest fires," *Journal of Field Robotics*, vol. 23, no. 3-4, pp. 165–184, 2006. <https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.20109>
- [31] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005. <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>
- [32] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016. <https://doi.org/10.1109/TR0.2016.2624754>
- [33] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon path planning for 3D exploration and surface inspection," *Autonomous Robots*, vol. 42, pp. 291–306, 2016. <https://link.springer.com/article/10.1007/s10514-016-9549-1>

Ringraziamenti

Questi anni di università sono stati caratterizzati da molti sacrifici, rinunce e momenti di sofferenza. Fortunatamente, queste difficoltà sono state sempre alleviate dalla presenza delle persone a me care. A tutti coloro che mi sono stati vicini, dico Grazie, dal cuore. Tuttavia, alcune persone meritano un ringraziamento particolare:

Ai miei genitori,

che, nonostante tutto, non mi hanno mai fatto mancare nulla. Nonostante i vari problemi familiari, mi hanno sempre dato la possibilità di fare ciò che desideravo, supportandomi nel bene e nel male. Grazie a loro, sono la persona forte di oggi. Mi hanno insegnato i valori della vita, tra cui l'educazione e la dedizione. Devono sapere che tutto ciò che faccio, tutto l'impegno che metto in ambito accademico e lavorativo, è per restituire loro, un giorno, tutto ciò che mi hanno dato, senza mai chiedere nulla in cambio.

A Elia,

per essere sempre stato vicino a me, nonostante il mio carattere molto particolare.

A mio cugino Matteo,

per essere il mio più grande punto di riferimento da sempre. Quando penso di non farcela, mi ricordo quanto è intelligente e mi motiva, cercando di diventare migliore di lui (è impossibile).

Alle mie carissime nonne,

per avermi sempre detto che sono il più bravo di tutti, per avermi cucinato per tutta la mia vita, per avermi fatto passare i momenti più belli della mia infanzia.

A Sara,

per essere entrata nella mia vita e avermi dato così tanto in così poco. Lei mi supporta qualsiasi cosa io faccia, mi fa analizzare i vantaggi e gli svantaggi delle mie scelte, mi ha fatto maturare tanto e mi sta dando moltissimo affetto. Potrei spendere mille parole per ringraziarla. Le vorrei solo dire che per me contano le piccole cose, come quando mi accompagna ad ogni mia gara, mi prepara da mangiare esattamente come lo voglio io e fa di tutto pur di farmi stare tranquillo. Non immagina quanto apprezzo e noto queste cose. Mi difende sempre e soprattutto mi aiuta ad affrontare i momenti difficili che magari, senza di lei, non riuscirei a superare.

A Nicco,

per essere un mio grande amico e un grande punto di riferimento. Con lui mi sono fatto le più grandi risate della mia vita. Deve sapere che ogni goccia di sudore che ho versato in questo periodo è stata per mantenere una promessa: sto arrivando a Torino.

A Dani,

perché il nostro rapporto è impossibile da descrivere a parole. Non penso sia suo volere che io spenda troppe parole per ringraziarlo, dato che passiamo già tanto tempo a discutere di argomenti che pochi ragazzi della nostra età affrontano. Tuttavia, una cosa mi sento di dirla: gli auguro il meglio e che tutti i momenti difficili che ha vissuto lo portino a raggiungere i suoi obiettivi. Io ci sarò sempre per lui.

A Matti,

perché ciò che abbiamo passato e creato insieme non ha eguali. Sono cresciuto e maturato con lui, realizzando i miei sogni insieme a lui. Spero che le nostre strade non si dividano mai, e ne sono convinto, perché il legame che abbiamo è come quello di una famiglia.

A Kri,

perché per primo ha creduto nei sogni miei e di Mattia, è stata la prima persona che ci ha dato tutta la sua passione e la sua fiducia. Sono sicuro che su di lui posso contare.

Alla mia amica Giorgia,

perché in questi anni è stata la persona da battere, in senso buono. È stata l'unica persona che, quando dovevo preparare la tesi e l'esame di Fisica 2 in sole tre settimane, si è messa a buttare giù delle idee mentre camminava, creando un vero e proprio piano di studio che nessun'altra persona al mondo potrebbe neanche sognarsi di portare a termine. Lei, nel suo, è la più forte di tutti.

Al Vernia,

per esserci sempre, indipendentemente dal problema. Mi ispiro molto a lui per quello che fa, nonostante le varie tempeste che deve affrontare, persiste senza mai mollare. Ne abbiamo passate tante, e le abbiamo superate tutte con le nostre forze. Spero che un giorno possa realizzare tutti i suoi sogni.

Al Sensei,

per essere stato una vera e propria guida, un pozzo di informazioni e di conoscenza. Una delle persone più intelligenti che abbia mai conosciuto.

Ad Albe,

per essere la persona più buona e genuina del mondo. Nessuno è sincero (e pollo) come lui. Per le giornate a casa sua, a studiare 10 ore al giorno, per le giornate al mare.

Alla mia amica Sara,

per aver sempre riconosciuto in me quel qualcosa in più, sin da quando avevo appena sedici anni. Anche lei è un grande punto di riferimento per me, con la sua dedizione e la sua determinazione in tutto ciò che fa. Nessuno può fermarla.

Ad Alessandro Palano,

per avermi aiutato a trovare quei maledetti video di fisica 1 e per essere stato fondamentale nella risoluzione di un problema di programmazione.

Ai tre Babu,

per essere degli amici speciali, quei veri amici con cui posso sempre essere me stesso. Grazie per avermi sempre ricordato di non portare sulle spalle il peso dei miei presunti errori, per avermi fatto capire che errori non erano. Grazie per avermi detto di trovare pace e di smettere di tormentarmi.

Ai futuri ingegneri pazzi

per avermi accompagnato in questo fantastico percorso, che hanno reso speciale.

A Calisthenics Florence ♥ ,

per essere una vera e propria famiglia, per essere una delle mie più grandi fonti di felicità. Ringrazio tutti coloro che ne fanno parte, che ne portano avanti i valori e che ci credono. Per me significa davvero tanto. Porteremo in alto questo nome e non ci fermeremo. AU AU AU, TO THE TOP NEVER STOP.

Ivan Necerini

